

AUTOMATED CYBER DEFENSE WITH GENERALIZABLE GRAPH-BASED REINFORCEMENT LEARNING AGENTS

Isaiah J. King^{1,2}, Benjamin Bowman¹, & H. Howie Huang^{1,2}

¹Cybermonic LLC, ²The George Washington University GraphLab
{isaiah, benjamin, howie}@cybermonic.com

ABSTRACT

Deep reinforcement learning (RL) is emerging as a viable strategy for automated cyber defense (ACD). The traditional RL approach represents networks as a list of computers in various states of safety or threat. Unfortunately, these models are forced to overfit to specific network topologies, rendering them ineffective when faced with even small environmental perturbations. In this work, we frame ACD as a two-player context-based partially observable Markov decision problem with observations represented as attributed graphs. This approach allows our agents to reason through the lens of relational inductive bias. Agents learn how to reason about hosts interacting with other system entities in a more general manner, and their actions are understood as edits to the graph representing the environment. By introducing this bias, we will show that our agents can better reason about the states of networks, and zero-shot adapt to new ones. We show that this approach outperforms the state-of-the-art by a wide margin, and makes our agents capable of defending never-before-seen networks against a wide range of adversaries in a variety of complex, and multi-agent environments.

1 INTRODUCTION

Automated cyber defense (ACD) systems are agents which, with no human intervention, defend a network from complex cyber-attacks—automated, or human (Vyas et al., 2023). Like an autonomous security operations center, ACD agents monitor the network at all times, waiting to respond to a cyber-attack. When an incident occurs, the ACD agent would have the power to update firewall rules, reset machines, etc., to prevent the intruder from spreading. Using a simulated network environment, we can train agents for this task using reinforcement learning (RL). Prior works in this field compress the environment into a vector and feed it into deep RL networks (Chai et al., 2020; Chowdhary et al., 2021; Eghtesad et al., 2020; Piplai et al., 2022; Ridley, 2018). This naive approach produces agents that are adept at defending the network they were trained on, but overly sensitive to minor changes in the network topology. As such, these agents are rarely evaluated in new and/or modified environments, a situation that is essential for any technology to be viable in the real world.

This observation coincides with a very old problem in RL: environmental overfitting (Whiteson et al., 2011). In traditional supervised learning, a model has become overfit if it memorizes every data point and is unable to generalize about data it has never seen. To account for this, one would partition the data into disjoint training and testing sets to ensure the model can adapt to uncertainty. In RL, the uncertainty lies in how an action will affect the environment. Environmental overfitting occurs when an agent no longer has uncertainty about the world it interacts with. Admittedly, model overfitting is not a problem in some cases; most RL benchmarks train and evaluate on the same environments (Kirk et al., 2023), and why shouldn’t they? An agent that plays chess can always expect the game to start with the same two sets of pieces placed in the same way across the same sixty-four squares. There is no reason to expect a chess-playing agent to adapt to a 9×9 board with five knights. But, for many real-world systems, such as ACD, the assumption that the environment will never change does not hold (Almasan et al., 2022).

One way to represent differences between networks’ topologies and encourage agents to learn policies that generalize across environments is to represent them as graphs. The use of graphs for state representation has been done by several prior works in this field (Ridley, 2018; Collyer et al.,

自动化网络防御与通用图强化学习代理

以赛亚·J·金^{1,2}, 本杰明·鲍曼¹, & H·黄^{1,2}网络monic公司, ² 乔治华盛顿
顿大学图实验室{isaiah, benjamin, howie}@cybermonic.com

摘要

深度强化学习（RL）正逐渐成为自动化网络防御（ACD）的一种可行策略。传统的RL方法将网络表示为处于不同安全或威胁状态下的计算机列表。然而，这些模型被迫过度拟合特定的网络拓扑，导致它们在面对微小环境扰动时无效。在这项工作中，我们将ACD构建为一个双玩家基于上下文的非完全可观察马尔可夫决策过程，其中观察以属性图的形式表示。这种方法使我们的代理能够从关系归纳偏差的角度进行推理。代理学习以更通用的方式推理主机与其他系统实体交互的方法，而它们的行动则被理解为对表示环境的图的编辑。通过引入这种偏差，我们将证明我们的代理能够更好地推理网络状态，并对新的网络进行零样本适应。我们证明这种方法大幅优于现有技术，并使我们的代理能够在各种复杂的多代理环境中，防御从未见过的新网络，对抗广泛的对手。

1 简介

自动化网络防御（ACD）系统是无需人工干预的代理，用于防御网络免受复杂的网络攻击——无论是自动攻击还是人为攻击（Vyas 等人，2023）。像自主安全运营中心一样，ACD 代理全天候监控网络，等待响应网络攻击。当发生事件时，ACD 代理将有权更新防火墙规则、重置机器等，以阻止入侵者扩散。使用模拟网络环境，我们可以通过强化学习（RL）训练代理执行此任务。该领域的先前工作将环境压缩为向量，并将其输入深度强化学习网络（Chai 等人，2020; Chowdhary 等人，2021; Eghtesad 等人，2020; Piplai 等人，2022; Ridley，2018）。这种简单的方法产生的代理擅长防御其训练的网络，但对网络拓扑的微小变化过于敏感。因此，这些代理很少在新的和/或修改过的环境中进行评估，而这种情况对于任何技术要在现实世界中发挥作用至关重要。

这一观察结果与强化学习（RL）中一个非常古老的问题相吻合：环境过拟合（Whiteson 等人，2011）。在传统的监督学习中，如果模型记住了所有数据点且无法泛化到从未见过的数据，则认为其过拟合。为了解决这个问题，人们会将数据划分为互不相交的训练集和测试集，以确保模型能够适应不确定性。在强化学习中，不确定性在于一个动作将如何影响环境。当智能体不再对其交互的世界存在不确定性时，就会发生环境过拟合。诚然，在某些情况下，模型过拟合并非问题；大多数强化学习基准都在同一环境中进行训练和评估（Kirk 等人，2023），这又有什么不对呢？一个下棋的智能体总是可以预期游戏将从相同的方式开始，在相同的六十四个方格上放置相同的两套棋子。没有必要期望一个下棋的智能体能够适应一个有五枚骑士的棋盘。但是，对于许多现实世界系统，例如 ACD，环境永远不会变化的假设并不成立（Almasan 等人，2022）。

一种表示网络拓扑差异并鼓励智能体学习跨环境泛化策略的方法是将其表示为图。该领域已有若干先前工作使用图进行状态表示（Ridley, 2018; Collyer 等人，

2022; Gangupantulu et al., 2021), but none process the graph directly. Agents in prior work receive graphs representing the environment as part of their observations, but when processing them for their agents, the graph is compressed into a vector, and information is lost. Most often, the topological structure of the graph is thrown out entirely, and information about individual nodes is concatenated together (Piplai et al., 2022; Ridley, 2018; Booker & Musman, 2020; Foley et al., 2022; Walter et al., 2021; Wolk et al., 2022). This results in models that learn about implicit relationships between nodes but cannot adapt to explicit changes of novel network topologies because these changes cannot be communicated to the agents. Perturbations as simple as changing the order in which nodes are indexed can cause these models to perform no better than random.

In this work, we propose a novel, graph-centric strategy for highly generalizable automated network defense agents. Harnessing the power of relational inductive bias (Battaglia et al., 2018), we provide our models with the full graph of the network, which they process without compression using a graph neural network (GNN) (Kipf & Welling, 2016). Inductive GNNs process graph input in a permutation-invariant manner and are structurally unable to rely on fixed node identities or positions. Automorphisms between nodes are implicitly understood to GNN-based models as features and edges are identical regardless of node mapping. However, this is not the case with traditional models as the columns of their parameters are fixed, such that automorphic node permutations do not produce identical outputs. The output of this process is a matrix of node representations, called embeddings, which contain information about each node’s features, as well as the features of their k -hop neighborhood. The node embeddings are then used as inputs to a policy network that selects the best action. Importantly, actions are not formulated as a fixed-length list, as is done by prior works, including graph-based approaches (Janisch et al., 2020); instead, we represent actions as functions upon individual nodes in the graph. This allows for changing topologies and changing action spaces. Adding an additional host to a network means adding several more actions relating to the defense of that host. For tabular methods, this would require fully retraining the agent; with our method, the action space is already a function of the graph size, so no retraining is required.

This work presents a generalizable framework for graph-based RL that allows for actions upon nodes and edges, with variably sized graphs. To the best of our knowledge, our approach is the first time relational inductive bias has been applied to ACD. We will show that our approach finds defense strategies that generalize better than prior work in the same field. The generality afforded by relational inductive bias means our agents can be deployed to new environments without retraining, saving potentially days of training time. In a simulated network environment, our agents score more than 4x higher than prior works and maintain that high score across environments with varied numbers of hosts—something the prior works are unable to do at all. In more complex environments, we show that when we perturb the topology or introduce new adversaries, our models perform better than all prior works. Finally, we show that our approach is also applicable to multi-agent reinforcement learning problems. Ours is currently the highest performing non-heuristic policy in the CC4 (TTCP CAGE Working Group, 2023) environment. The source code for the agents and experiments is available at <https://github.com/cybermonic/ACD-With-GraphRL>

2 RELATED WORK

Relational Inductive Bias: As in traditional machine learning, an overfit RL agent will optimize its policy to the random noise in the environment rather than its true distribution. This reflects a high degree of variance in the model. To counter overfitting, one must increase the model’s bias (Geman et al., 1992). Inductive bias helps models constrain their search space and reason with more generalized approaches (Mitchell, 1980). Inductive bias can be created implicitly via the choice of neural architecture, or explicitly by constraining the data the model receives. Motivated by the strong arguments of Battaglia et al. (2018) in favor of relational inductive bias, as well as the numerous positive results from empirical studies (Hamrick et al., 2018; Janisch et al., 2020; Khalil et al., 2017; Wang et al., 2018) we represent the networks we wish to defend as graphs.

RL for Automated Cyber Defense: There exist many agents trained for cyber-tasks that focus on individual entities within networks, or abstract networks into individual units to defend (Eghesad et al., 2020; Wu et al., 2018). When networks are represented as graphs, they are commonly abstracted into tabular representations for simplicity (Booker & Musman, 2020; Foley et al., 2022; Applebaum et al., 2022). Works that include graph representations often focus on finding vulner-

2022; Gangupantulu 等人, 2021), 但它们都不直接处理图。先前工作中的智能体接收表示环境的图作为其观察的一部分, 但在处理这些图时, 图会被压缩成一个向量, 导致信息丢失。通常情况下, 图中的拓扑结构会被完全丢弃, 而关于单个节点的信息会被连接在一起 (Piplai 等人, 2022; Ridley, 2018; Booker & Musman, 2020; Foley 等人, 2022; Walter 等人, 2021; Wolk 等人, 2022)。这导致模型只能学习节点之间的隐式关系, 而无法适应新网络拓扑的显式变化, 因为这些变化无法传达给智能体。即使是简单的改变节点索引的顺序这种扰动, 也会导致这些模型的性能不优于随机水平。

在这项工作中, 我们提出了一种新颖的、以图为中心的策略, 用于开发高度泛化的自动化网络防御代理。我们借助关系归纳偏置 (Battaglia 等人, 2018) 的力量, 向模型提供整个网络图, 它们使用图神经网络 (GNN) (Kipf & Welling, 2016) 对其进行处理, 而无需压缩。归纳 GNN 以置换不变的方式处理图输入, 并且在结构上无法依赖固定的节点身份或位置。节点之间的自同构关系被隐式地理解为基于 GNN 的模型, 因为无论节点映射如何, 特征和边都是相同的。然而, 传统模型则不同, 因为它们的参数列是固定的, 因此自同构节点置换不会产生相同的输出。该过程的输出是一个节点表示矩阵, 称为嵌入, 其中包含每个节点的特征信息, 以及它们各自的 k 跳邻域的特征。然后, 节点嵌入被用作策略网络输入, 该网络选择最佳动作。重要的是, 动作不是像先前工作那样表示为固定长度的列表, 包括基于图的方法 (Janisch 等人, 2020); 相反, 我们将动作表示为对图中单个节点的函数。这使得拓扑结构和动作空间都可以改变。向网络添加一个额外的主机意味着添加更多与该主机防御相关的动作。对于表格方法, 这需要完全重新训练代理; 而我们的方法中, 动作空间已经是图大小的函数, 因此无需重新训练。

这项工作提出了一种通用的基于图的强化学习框架, 该框架允许对节点和边进行操作, 并支持可变大小的图。据我们所知, 我们的方法首次将关系归纳偏差应用于 ACD。我们将证明, 我们的方法能够找到比同一领域先前工作更泛化的防御策略。关系归纳偏差带来的泛化能力意味着我们的智能体可以在新环境中部署而无需重新训练, 从而节省潜在的数天训练时间。在一个模拟的网络环境中, 我们的智能体得分比先前工作高出 4 倍以上, 并在具有不同主机数量的环境中保持这一高分——而先前工作完全无法做到这一点。在更复杂的环境中, 我们展示了当我们对拓扑进行扰动或引入新的对手时, 我们的模型表现优于所有先前工作。最后, 我们证明我们的方法也适用于多智能体强化学习问题。目前, 我们的方法在 CC4 (TTCP CAGE 工作组, 2023) 环境中表现最佳的非启发式策略。智能体和实验的源代码可在 <https://github.com/cybermonic/ACD-With-GraphRL> 获取。

2 相关工作

关系归纳偏差: 与传统机器学习类似, 过度拟合的强化学习智能体会优化其策略以适应环境中的随机噪声, 而不是其真实分布。这反映了模型中高度的方差。为了对抗过度拟合, 必须增加模型的偏差 (Geman 等人, 1992)。归纳偏差有助于模型约束其搜索空间, 并以更通用的方法进行推理 (Mitchell, 1980)。归纳偏差可以通过选择神经网络架构隐式地创建, 或通过约束模型接收的数据显式地创建。受 Battaglia 等人 (2018) 强烈支持关系归纳偏差的论证, 以及来自实证研究的众多积极结果 (Hamrick 等人, 2018; Janisch 等人, 2020; Khalil 等人, 2017; Wang 等人, 2018) 的启发, 我们将希望防御的网络表示为图。

RL 用于自动化网络防御: 存在许多针对网络中单个实体或抽象网络为单个单元进行防御的智能体 (Eghesad 等人, 2020; Wu 等人, 2018)。当网络表示为图时, 通常为了简化而抽象成表格表示 (Booker & Musman, 2020; Foley 等人, 2022; Applebaum 等人, 2022)。包含图表示的研究工作通常专注于寻找漏洞

abilities (Piplai et al., 2022; Yousefi et al., 2018) or automated penetration testing (Gangupantulu et al., 2021; Walter et al., 2021). While many graph-based ACD agents exist in the literature, they are often constrained in unrealistic ways. Prior works, (Chai et al., 2020; Chowdhary et al., 2021; Ridley, 2018; Cam, 2020) all consider the graph structure of networks, and train DQNs to defend them. However, they all fail to consider what happens if nodes are added or removed, or even if the order in which nodes are indexed changes. More relevant to our work are the ACD approaches which highlight the importance of generalization to new topologies. One method to achieve this is aggregation. Gao et al. (2021) create a generalizable agent by creating an action space that is independent of node count and order. Instead, when an action is selected, it is applied to the node that can utilize it most efficiently according to a value function. Similarly, Gao & Wang (2021) aggregate the entirety of the network into a single entity to be defended; their observations are the network’s state in aggregate across k timesteps. However, it would be difficult to directly apply either of these approaches to the more general ACD environments we evaluate, as they are both specialized for narrow threat models. The most similar prior work to ours is Doorman et al. (2022). They model the network as a graph and train a DQN to select triplets of nodes to rewire for network hardening. Their approach is specific to this one task, however, and as we will show, it does not generalize well to more varied action spaces, but it is notable in that it is fully inductive.

3 BACKGROUND

A graph is a set of discrete objects $\mathcal{V}$ called nodes, and a set of edges $\mathcal{E} = \{(u, v) \mid u, v \in \mathcal{V}\}$ denoting relationships between them. An attributed graph is a graph with features associated with each node, $f : \mathcal{V} \rightarrow \mathbb{R}^d$. We denote these features as $\mathbf{X}$. Finally, we define node embeddings as the output of a function $\Phi(\mathcal{V}, \mathcal{E}, \mathbf{X})$ that processes the graph to encode node features and topology into a single vector. This function can be transductive, which means it can only produce embeddings for nodes it has seen during training, or it may be inductive, which means it can produce embeddings for nodes it has never seen before. We will evaluate both kinds of embedding functions.

A partially observable Markov decision process (POMDP) is defined as the 7-tuple

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{R}, \mathcal{T}, \phi, p \rangle. \quad (1)$$

Here, $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, and $p(s_0)$ is the distribution of possible starting states. $\mathcal{T}(s' \mid s, a)$ is the possibly stochastic transition function that determines the next state given a state and action. $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is a scalar reward function. The set $\mathcal{O}$ is the observation space, which is derived from the true state via the function $\phi : \mathcal{S} \rightarrow \mathcal{O}$. Any policy acting upon $\mathcal{M}$ will only observe the output of ϕ .

Kirk et al. (2023) define context-based POMDPs (CMDPs) as the set of all possible POMDPs in a space conditioned by a parameter sampled from distribution $\mathcal{C}$. This parameter changes the possible state space, transition probabilities, and rewards, but not the action space. If we divide the context space into disjoint sets, $\mathcal{M}|_{\mathcal{C}_{\text{train}}}$ and $\mathcal{M}|_{\mathcal{C}_{\text{test}}}$, the objective is to find policy π which maximizes the reward over CMDPs parameterized from the test space conditioned only on experiences from the training space.

4 GRAPH-BASED REINFORCEMENT LEARNING

In this section, we will describe the architecture of our graph-analytic models. We will first describe how we represent observations as graphs and actions as graph edits. Next, we describe the three models we evaluated: one transductive model, and two inductive models. Transductive models assume the size and node-ordering of a graph is fixed, while inductive models can adapt to unseen environments, with varying numbers of nodes. We used GCN as the GNN architecture, but this approach is agnostic to the specific choice of GNN.¹

¹Experiments showed low variance between different choices of GNN architectures. We provide an ablation study demonstrating this in Appendix B.

先前的工作 (Chai 等人, 2020; Chowdhary 等人, 2021; Ridley, 2018; Cam, 2020) 都考虑了网络的结构, 并训练深度Q网络来防御它们。然而, 它们都没有考虑如果节点被添加或删除, 或者节点索引的顺序发生变化会发生什么。与我们工作更相关的是强调泛化到新拓扑结构重要性的自动化网络防御方法。实现这一目标的一种方法是聚合。Gao 等人 (2021) 通过创建一个与节点数量和顺序无关的动作空间来创建一个可泛化的智能体。相反, 当选择一个动作时, 它会被应用于能够最有效地利用该动作的节点, 根据价值函数。类似地, Gao & Wang (2021) 将整个网络聚合为一个单一实体进行防御; 他们的观察是网络在 k 个时间步长内的聚合状态。然而, 将这些方法直接应用于我们评估的更通用的自动化网络防御环境将非常困难, 因为它们都针对狭窄的威胁模型进行了专门设计。与我们最相似的先前工作是 Doorman 等人 (2022)。他们将网络建模为图, 并训练一个深度Q网络来选择要重新连接的节点三元组以增强网络。然而, 他们的方法仅针对这一特定任务, 正如我们将要展示的, 它并不能很好地泛化到更多样化的动作空间, 但值得注意的是, 它是完全归纳式的。

3 背景

图是一组离散对象 $\mathcal{V}$, 称为节点, 以及一组表示它们之间关系的边 $\mathcal{E} = \{(u, v) \mid u, v \in \mathcal{V}\}$ 。带属性的图是每个节点带有特征的图 $f : \mathcal{V} \rightarrow \mathbb{R}^d$ 。我们将这些特征表示为 $\mathbf{X}$ 。最后, 我们定义节点嵌入为函数 $\Phi(\mathcal{V}, \mathcal{E}, \mathbf{X})$ 的输出, 该函数处理图以将节点特征和拓扑编码为单个向量。这个函数可以是同态的, 这意味着它只能为训练期间见过的节点产生嵌入, 或者它可能是归纳的, 这意味着它可以产生它之前从未见过的节点的嵌入。我们将评估这两种类型的嵌入函数。

部分可观察马尔可夫决策过程 (POMDP) 定义为 7 元组

$$\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{R}, \mathcal{T}, \phi, p \rangle. \quad (1)$$

在这里, $\mathcal{S}$ 是状态集, $\mathcal{A}$ 是动作集, $p(s_0)$ 是可能起始状态的分布。 $\mathcal{T}(s' \mid s, a)$ 是可能的随机转移函数, 它根据状态和动作确定下一个状态。 $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ 是标量奖励函数。集合 $\mathcal{O}$ 是观察空间, 它通过函数 $\phi : \mathcal{S} \rightarrow \mathcal{O}$ 从真实状态导出。任何作用于 $\mathcal{M}$ 的策略都只会观察到 ϕ 的输出。

Kirk 等人 (2023) 将基于上下文的 POMDPs (CMDPs) 定义为在由从分布 $\mathcal{C}$ 中采样的参数条件化的空间中所有可能的 POMDPs 的集合。该参数会改变可能的状态空间、转换概率和奖励, 但不会改变动作空间。如果我们把上下文空间划分为不相交的集合 $\mathcal{M}|_{\mathcal{C}_{\text{train}}}$ 和 $\mathcal{M}|_{\mathcal{C}_{\text{test}}}$, 那么目标就是在仅由训练空间经验条件化的测试空间参数化的 CMDPs 中找到最大化奖励的策略 π 。

4 基于图的强化学习

在本节中, 我们将描述我们的图分析模型的架构。我们将首先描述如何将观察结果表示为图, 以及如何将动作表示为图编辑。接下来, 我们将描述我们评估的三个模型: 一个转导模型和两个归纳模型。转导模型假设图的规模和节点顺序是固定的, 而归纳模型可以适应未见过的环境, 节点数量可变。我们使用GCN作为GNN架构, 但这种方法对GNN的具体选择是无关紧要的。¹

¹实验表明, 不同的 GNN 架构选择之间方差较低。我们在附录 B 中提供了一个消融研究来证明这一点。

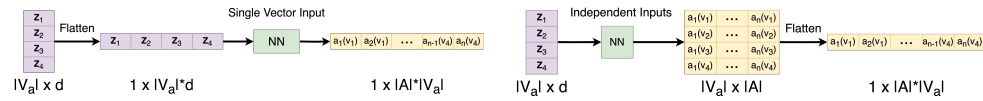


Figure 1: Transductive Actor

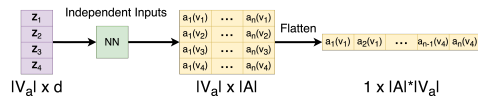


Figure 2: Inductive Actor

4.1 STATE-ACTION SPACE

States: In all experiments, we use individual hosts as nodes, and forms of inter-host communication as edges. Other entities within a network can also be modeled as nodes in a graph, with their node type specified as one of their node features. Unlike prior works which use features derived from graphs as their observations (Collyer et al., 2022; Wolk et al., 2022), we use the attributed graph itself. This means agents’ observations are represented by the 3-tuple $\mathcal{O} = \langle \mathcal{V}, \mathcal{E}, \mathbf{X} \rangle$, representing nodes, edges, and node features, respectively. Importantly, we do not assume the agent has full visibility into the network. The graph provided in the observation may have missing information about whether hosts are compromised, the possible paths an attacker could use to pivot through the network, etc., such that $\mathcal{O} \subseteq \mathcal{S}$.

Actions: Following the model of Janisch et al. (2020), we model each game as a system of objects (or nodes) that have relationships (or edges) with one another. Actions can be performed upon some subset of nodes to change the environment. Thus, for each actionable node $v_i \in \mathcal{V}_a \subseteq \mathcal{V}$, given an action space $\mathcal{A} = \{a_0, \dots, a_n\}$, actions are represented as functions upon those nodes, $a_n(v_i)$. This makes the total action space for an environment $\mathcal{A} \otimes \mathcal{V}_a$. By representing actions as functions upon discrete objects, rather than a fixed array as is done by tabular methods (Watkins & Dayan, 1992), we are free to increase or decrease the action space without retraining by changing the size of set $\mathcal{V}_a$.

The results of actions produce graph edits. These edits may be in the form of node or edge additions or deletions, or changes in nodes’ features. For example, the action $a_n(v_i)$ may create a node v_j with an edge to v_i ; it could change the feature vector x_i associated with node v_i ; or it could remove the node v_i from the graph entirely. With this abstraction, we can further extend the action space to include edge-level actions as a function of $a_n(v_i, v_j)$, upon actionable edges $\mathcal{E}_a$. This work considers environments that have node-level and edge-level actions, $\mathcal{A} = \mathcal{A}_V \cup \mathcal{A}_E$.

4.2 AGENT DESIGN

To combine the information about the graph structure, and its features, we use a graph neural network (GNN) (Scarselli et al., 2008) to produce node embeddings. We employ a graph convolutional network (GCN) (Kipf & Welling, 2016) for its expressiveness, and inductive abilities. The defining function of the GCN is

$$\mathbf{H}^{(\ell+1)} = \sigma(\mathbf{D}^{\frac{1}{2}} \mathbf{A} \mathbf{D}^{\frac{1}{2}} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \quad (2)$$

where $\mathbf{A}$ is the adjacency matrix of the graph plus $\mathbf{I}$. $\mathbf{D}$ is the degree of each node, and $\mathbf{W}^{(\ell)}$ and $\mathbf{b}^{(\ell)}$ are trainable parameters. For each node in the graph, this function averages the features of its neighbors, then passes the output through a single fully connected nonlinear neural network layer. Using k GCN layers will encode information about each node’s k -hop neighborhood. In practice, we use the more efficient message-passing paradigm implemented by PyTorch Geometric (Fey & Lenssen, 2019).

The agent learns via Proximal Policy Optimization (PPO) (Schulman et al., 2017). Both the actor-and critic-networks create node embeddings $\mathbf{Z}$ using 2-layer GCNs. The embeddings are then passed through additional fully connected layers to produce the probability distribution function, or the state value estimate for the actor and critic, respectively. We evaluate three methods to convert from node embeddings to action probabilities. Using the terminology of graph representation learning, we refer to these approaches as either inductive or transductive (Hamilton et al., 2017).

Transductive models operate as prior tabular RL approaches do. As shown in Figure 1, they concatenate the node embeddings into a single, fixed-sized vector, and pass it through a traditional neural network to produce a single vector of size $|\mathcal{V}_a| \cdot |\mathcal{A}|$. This approach is not generalizable to new

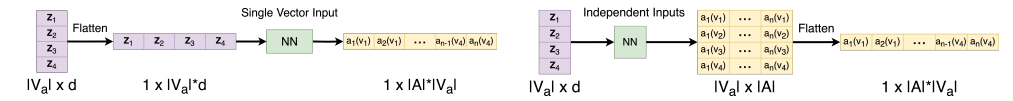


图1：归纳式演员

图2：演绎式演员

4.1 状态-动作空间

状态：在所有实验中，我们使用单个主机作为节点，并将主机间通信形式作为边。网络中的其他实体也可以作为图中的节点进行建模，其节点类型指定为其节点特征之一。与先前使用从图中派生的特征作为观察结果的工作（Collyer等人，2022年；Wolk等人，2022年）不同，我们使用带属性的图本身。这意味着代理的观察结果由三元组 $\mathcal{O} = \langle \mathcal{V}, \mathcal{E}, \mathbf{X} \rangle$ 、表示，分别代表节点、边和节点特征。重要的是，我们不假设代理对网络有完全的可见性。观察中提供的图可能缺少关于主机是否被攻破、攻击者可能用来在网络中横向移动的可能路径等信息，因此 $\mathcal{O} \subseteq \mathcal{S}$ 。

动作：遵循Janisch等人（2020）的模型，我们将每个游戏建模为一个由对象（或节点）组成的系统，这些对象之间具有关系（或边）。动作可以对节点的一个子集执行以改变环境。因此，对于每个可执行节点 $v_i \in \mathcal{V}_a \subseteq \mathcal{V}$ ，给定一个动作空间 $\mathcal{A} = \{a_0, \dots, a_n\}$ ，动作被表示为对这些节点的函数， $a_n(v_i)$ 。这构成了环境 $\mathcal{A} \otimes \mathcal{V}_a$ 的总动作空间。通过将动作表示为离散对象的函数，而不是表格方法（Watkins & Dayan, 1992）所使用的固定数组，我们可以通过改变集合 $\mathcal{V}_a$ 的大小来自由增加或减少动作空间，而无需重新训练。

动作的结果产生图编辑。这些编辑可能以节点或边的添加或删除的形式出现，或者改变节点的特征。例如，动作 $a_n(v_i)$ 可能会创建一个连接到 v_i 的节点 v_j ；它可能改变与节点 v_i 相关联的特征向量 x_i ；或者它可能完全从图中移除节点 v_i 。通过这种抽象，我们可以进一步扩展动作空间，以包括作为 $a_n(v_i, v_j)$ 函数的边级动作，作用于可执行边 $\mathcal{E}_a$ 。这项工作考虑了具有节点级和边级动作的环境， $\mathcal{A} = \mathcal{A}_V \cup \mathcal{A}_E$ 。

4.2 AGENT 设计

为了结合图结构及其特征的信息，我们使用图神经网络（GNN）（Scarselli 等人，2008）来生成节点嵌入。我们采用图卷积网络（GCN）（Kipf & Welling, 2016），因为它具有表达能力，以及归纳能力。GCN 的定义函数是

$$\mathbf{H}^{(\ell+1)} = \sigma(\mathbf{D}^{\frac{1}{2}} \mathbf{A} \mathbf{D}^{\frac{1}{2}} \mathbf{H}^{(\ell)} \mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}) \quad (2)$$

其中 $\mathbf{A}$ 是图的邻接矩阵加上 $\mathbf{I}$ 。 $\mathbf{D}$ 是每个节点的度数，而 $\mathbf{W}^{(\ell)}$ 和 $\mathbf{b}^{(\ell)}$ 是可训练参数。对于图中的每个节点，此函数对其邻居的特征进行平均，然后将输出通过一个单层的全连接非线性神经网络层。使用 k GCN 层将编码每个节点 k -跳邻域的信息。在实践中，我们使用 PyTorch Geometric (Fey & Lenssen, 2019) 实现的更高效的消息传递范式。

智能体通过近端策略优化（PPO）（Schulman 等人，2017）进行学习。行为者和评价者网络都使用 2 层 GCN 创建节点嵌入 $\mathbf{Z}$ 。然后，嵌入通过额外的全连接层传递，以生成概率分布函数，或行为者和评价者的状态值估计。我们评估了三种从节点嵌入转换为动作概率的方法。使用图表示学习的术语，我们将这些方法称为归纳或演绎（Hamilton 等人，2017）。

传递模型运作方式与早期的表格式强化学习方法类似。如图1所示，它们将节点嵌入拼接成一个固定大小的向量，并将其通过传统神经网络处理，生成一个大小为 $|\mathcal{V}_a| \cdot |\mathcal{A}|$ 的向量。这种方法无法推广到新的拓扑结构。然而，由于传递模型通常优于归纳模型（Lachaud 等人，2022年），它可作为展示该领域关系偏差最大潜力的基础。

topologies. However, as transductive models tend to outperform inductive ones (Lachaud et al., 2022), it serves as a basis to demonstrate the maximum potential of relational bias in this domain.

For inductive models, the embeddings for each node must be processed in an order- and length-invariant way. The simplest way to do this is illustrated in Figure 2. The model passes each node embedding independently through a fully connected layer, and the output is then flattened. The actor-network uses the softmax of the flattened output to directly calculate action probabilities (e.g., $v_{i,j}$ represents the log-odds of taking action j on node i). If more actionable nodes are added to the graph, the inductive GNN will output a $|\mathcal{V} \cup \mathcal{V}'| \times |\mathcal{A}|$ dimensional matrix, which can be interpreted in the same way without retraining, thus allowing for full inductivity. The critic-network, which attempts to evaluate the value of the current global state, projects each row of $\mathbf{Z}_{\mathcal{V}_a}$ into a single dimension, then pools the batch into a single value. This method, which we refer to as the *naive inductive model* is simplistic, but we will show it is also powerful. However, it has a major drawback: node embeddings do not have the context of other nodes' states if they are greater than k hops away.

To address this, we adopt a modified form of the attention-based node pooling proposed by Janisch et al. (2020). This modification occurs in the node embedding step of the model. In each layer of the GCN, the embeddings for all nodes $\mathbf{Z}$ are calculated. Then, the embeddings of actionable nodes, $\mathbf{Z}_{\mathcal{V}_a}$, are extracted and concatenated with a global graph state vector $\mathbf{g}$. We then calculate the updated global state vector of the graph as

$$\mathbf{g}' = \mathbf{g} + \phi_g \left(\text{POOL}_{i \in \mathcal{V}_a} \{ \phi_v(\mathbf{g}, \mathbf{z}_i) \cdot \phi_a(\mathbf{g}, \mathbf{z}_i) \} \right) \quad (3)$$

where ϕ_v , and ϕ_g are fully connected networks, and ϕ_a is a fully connected network with softmax activation. POOL represents any pooling function that is order-invariant. The original work suggests sum-pooling, but this can cause issues when testing in environments that have many more nodes than the training environment. In this work, we use mean- or max-pooling to address this problem.

The actor-network concatenates the final $\mathbf{g}$ vector to each of the final node embeddings, then uses the final vectors in the inductive method we previously described. This ensures that each node embedding contains information about all other nodes in the network, which allows them to weigh the importance of taking an action upon themselves, vs. an action somewhere else in the network. The critic network, as it is calculating the global value of the network's state, uses the $\mathbf{g}$ vector directly as input. We refer to this method as the *attention inductive model*.

Inductive models also support *edge-level actions*. We formulate the probability of taking an action on an edge as the output of an additional function that takes the source and destination node embeddings as input. In this work, to calculate the probability of taking an edge action, we calculate $f(\mathbf{Z}_{\text{src}} \odot \mathbf{Z}_{\text{dst}})$ where $f(\cdot)$ is a fully connected layer with $|\mathcal{A}_E|$ -dimensional output, $\odot$ represents the Hadamard product, and $\mathbf{Z}$ is the set of node embeddings such that $\langle \text{src}, \text{dst} \rangle \in \mathcal{E}_a$. The output of $f(\cdot)$ is then concatenated to the flattened probability vector of node-level actions.

5 EXPERIMENTS

We evaluate our agent in three environments of increasing complexity. In each environment, our agent (the blue agent) defends the network from an attacker (the red agent). The threat model between each environment varies, but broadly, we assume that the system can be modeled as a graph of hosts whose security states transition over time between "safe" and "compromised." These transitions are governed by both attacker and defender actions. The specific transition dynamics vary by environment but are always represented within the environment's MDP. We assume that the attacker begins with root access to a single host and can take actions that increase the likelihood of compromising additional hosts. The attacker operates under partial observability of the network, must scan to discover reachable hosts, and lacks prior knowledge of the topology. The defender also has partial observability, though the scope and quality of information vary by environment. In each timestep, both attacker and defender select actions simultaneously. The attacker's objective is to maximize the number of compromised hosts over a finite time horizon, while the defender aims to minimize this quantity.

Table 1 summarizes important details of each environment. For the first experiment, we analyze the Yawning Titan environment (Collyer et al., 2022), one of the first works to frame RL-based

然而，由于传递模型通常优于归纳模型（Lachaud 等人，2022年），它可作为展示该领域关系偏差最大潜力的基础。

对于归纳模型，每个节点的嵌入必须以不依赖于顺序和长度的方式进行处理。最简单的方法如图2所示。模型独立地将每个节点嵌入传递给全连接层，然后对输出进行展平。行动者网络使用展平输出的softmax直接计算行动概率（例如， $v_{i,j}$ 表示在节点 i 上采取行动 j 的对数概率）。如果向图中添加更多可行动的节点，归纳GNN将输出一个 $|\mathcal{V} \cup \mathcal{V}'| \times |\mathcal{A}|$ 维的矩阵，无需重新训练即可以相同的方式解释，从而实现完全的归纳性。评价网络尝试评估当前全局状态的价值，将 $\mathbf{Z}_{\mathcal{V}_a}$ 的每一行投影到单个维度，然后将批次聚合为单个值。我们称这种方法为朴素归纳模型，它虽然简单，但我们将证明它同样强大。然而，它有一个主要缺点：如果节点之间的距离大于 k 跳，节点嵌入将没有其他节点状态的信息。

为此，我们采用 Janisch 等人（2020）提出的基于注意力的节点池化方法的改进形式。这种修改发生在模型的节点嵌入步骤中。在 GCN 的每一层中，都会计算所有节点 $\mathbf{Z}$ 的嵌入。然后，提取可操作节点 $\mathbf{Z}_{\mathcal{V}_a}$ 的嵌入，并将其与全局图状态向量 $\mathbf{g}$ 连接。我们随后计算图更新的全局状态向量，

$$\mathbf{g}' = \mathbf{g} + \phi_g \left(\text{POOL}_{i \in \mathcal{V}_a} \{ \phi_v(\mathbf{g}, \mathbf{z}_i) \cdot \phi_a(\mathbf{g}, \mathbf{z}_i) \} \right) \quad (3)$$

其中 ϕ_v 、 ϕ_g 是完全连接的网络，而 ϕ_a 是带有 softmax 激活的完全连接网络。POOL 表示任何顺序不变的池化函数。原始工作建议使用求和池化，但在测试环境中节点数量远多于训练环境时，这可能会引发问题。在这项工作中，我们使用均值或最大池化来解决此问题。

行为者网络将最终的 $\mathbf{g}$ 向量连接到每个最终的节点嵌入，然后使用我们先前描述的归纳方法中的最终向量。这确保了每个节点嵌入都包含网络中所有其他节点的信息，从而允许它们权衡在自身上采取行动的重要性，与在网络其他地方采取行动的重要性。评价者网络在计算网络状态的全局值时，直接使用 $\mathbf{g}$ 向量作为输入。我们将此方法称为注意力归纳模型。

归纳模型也支持边缘级别的动作。我们将执行边缘动作的概率表述为额外函数的输出，该函数以源节点和目标节点嵌入为输入。在本工作中，为计算执行边缘动作的概率，我们计算 $f(\mathbf{Z}_{\text{src}} \odot \mathbf{Z}_{\text{dst}})$ ，其中 $f(\cdot)$ 是一个具有 $|\mathcal{A}_E|$ 维输出的全连接层， $\odot$ 表示哈达玛积，而 $\mathbf{Z}$ 是满足 $\langle \text{src}, \text{dst} \rangle \in \mathcal{E}_a$ 的节点嵌入集合，然后， $f(\cdot)$ 的输出与节点级别动作的扁平化概率向量连接起来。

5 实验

我们在三个复杂度递增的环境中评估我们的智能体。在每个环境中，我们的智能体（蓝色智能体）防御网络免受攻击者（红色智能体）的攻击。每个环境之间的威胁模型各不相同，但总体而言，我们假设系统可以建模为安全状态随时间在“安全”和“受感染”之间转换的主机图。这些转换受攻击者和防御者动作的共同影响。具体的转换动态因环境而异，但始终在环境的多智能体决策过程（MDP）中表示。我们假设攻击者最初拥有单个主机的根访问权限，并且可以采取增加感染更多主机可能性的动作。攻击者在网络的部分可观察性下运行，必须扫描以发现可达的主机，并且缺乏对拓扑结构的先验知识。防御者也有部分可观察性，但信息的范围和质量因环境而异。在每个时间步，攻击者和防御者同时选择动作。攻击者的目标是最大化在有限时间范围内受感染主机的数量，而防御者的目标是最小化这个数量。

表1总结了每个环境的详细信息。在第一个实验中，我们分析了Yawning Titan环境（Collyer等人，2022年），这是最早将基于RL的网络防御问题框架化为图问题之一的工作。

network defense as a graph problem.² Next, we evaluate on the CAGE Challenge 2 (CC2) environment (CAGE, 2022), which has a more complex action space and a more fine-grained reward function. Finally, we test our agent in the very complex CC4 environment (TTCP CAGE Working Group, 2023). In addition to being the largest network we test our agent on, it is also a multi-agent RL environment. Due to the page limit, please refer to the appendix for additional details on training configurations, ablation studies, and the details of the testing environments.

Table 1: Evaluation Environments

	Network Size	Per-node actions	Per-edge actions	Blue Agents	Red Agents
Yawning Titan	10-100	3	0	1	1
CC2	13	10	0	1	1
CC4	32-128	4	2	5	1-5

5.1 NETWORK SIMULATION WITH YAWNING TITAN ENVIRONMENT

This environment is a two-player CMDP where a heuristic red agent attempts to spread through a computer network, and our model attempts to defend it. The computer network is represented as a random Erdős-Rényi graph (Erdős et al., 1960). Each node represents a computer, and edges represent their ability to communicate. Details of the experimental setup and the state-action space of this environment are provided in the Appendix.

We use the same training configuration as the original work (Collyer et al., 2022). We train models for 5 million steps on a single random graph, then sample 50 new graphs and evaluate the models in these new environments without retraining for 10 episodes each. We compare our *Transductive*, *Naive*, and *Attention Inductive* models to the two approaches from the prior work: *Standard Observations* and *Graph Observations*. We also evaluate the architecture proposed by Doorman et al. (2022): a struc2vec-based GNN (Ribeiro et al., 2017) that uses sum aggregation to produce a global graph vector that is used in a similar way to our self-attention model. Their original approach was only designed for environments with a single action, but we modified the output to produce likelihoods for $|\mathcal{A}|$ actions per-node. Their architecture is similar to ours, but their approach uses a DQN rather than PPO. It is included here to compare the utility of both RL strategies for this problem. Table 2 reports the average reward of all 500 episodes, normalized such that 1.0 is a theoretically perfect score if no hosts were compromised at all.

Table 2: Mean score on random environments (higher is better)

	$ \mathcal{V} = 10$	$ \mathcal{V} = 20$	$ \mathcal{V} = 40$
Standard Observations	0.2511 ± 0.017	0.2310 ± 0.013	0.1683 ± 0.010
Graph Observations	0.2090 ± 0.005	0.3037 ± 0.013	0.2047 ± 0.013
Doorman et al. (2022)	0.7379 ± 0.011	-0.0385 ± 0.011	-0.0300 ± 0.007
Transductive	0.4246 ± 0.011	-0.1327 ± 0.000	-0.0966 ± 0.000
Naive Inductive	0.2681 ± 0.036	0.5439 ± 0.012	0.9308 ± 0.004
Attention Inductive	0.8167 ± 0.009	0.5634 ± 0.012	0.9955 ± 0.002

Unsurprisingly, the transductive model is unable to generalize to new environments. It is likely that it overfit to the node ordering it observed during training. Both the naive and attention-based inductive models more than double the tabular methods’ scores. When compared to the other GNN-based technique, as environments grew more complex, the model became less able to generalize to new graphs. Despite achieving scores comprable to our models during training, when evaluated on the new graphs, the Doorman et al. (2022) models failed to generalize. This may be explained by PPO’s greater exploration ability, allowing it to find more optimal policies in more complex environments compared to DQN De La Fuente & Guerra (2024). It could also be that the prior work’s choice of sum for the readout function in their model did not generalize to graphs with varied densities, leading to oversmoothing. We also observe that when $|\mathcal{V}| = 40$, the attention-based model appears to have found a nearly perfect strategy. We hypothesize that because it was trained in a more complex environment, the observations it received during training were more varied, and forced the model to find a more general policy, while models in smaller environments became more overfit.

²We select this work rather than Ridley (2018) because it has an additional focus on domain generalization.

接下来，我们在CAGE挑战2（CC2）环境（CAGE，2022年）上进行评估，该环境具有更复杂的动作空间和更细粒度的奖励函数。最后，我们在非常复杂的CC4环境（TTCP CAGE工作组，2023年）中测试我们的智能体。除了是我们测试智能体的最大网络之外，它还是一个多智能体RL环境。由于页数限制，有关训练配置、消融研究和测试环境详细信息的更多信息，请参阅附录。

表1：评估环境

	网络规模	每节点操作	每边操作	蓝色代理	红色代理
打哈欠的巨人	10-100	3	0	1	1
CC2	13	10	0	1	1
CC4	32-128	4	2	5	1-5

5.1 使用YAWNINGTITAN环境的网络模拟

该环境是一个双人CMDP，其中启发式红色智能体试图通过计算机网络传播，而我们的模型试图进行防御。计算机网络表示为一个随机的Erdős-Rényi图（Erdős等人，1960年）。每个节点代表一台计算机，边代表它们通信的能力。实验设置的详细信息和该环境的状态-动作空间在附录中提供。

我们使用与原始工作（Collyer等人，2022年）相同的训练配置。我们在单个随机图上训练模型500万步，然后采样50个新图，并在这些新环境中评估模型，每个环境不重新训练10个回合。我们将我们的Transductive、Naive和Attention Inductive模型与先前工作中的两种方法进行比较：标准观察和图观察。我们还评估了Doorman等人（2022年）提出的架构：一个基于struc2vec的GNN（Ribeiro等人，2017年），它使用求和聚合来生成一个全局图向量，该向量以与我们的自注意力模型类似的方式使用。他们的原始方法仅设计用于具有单个动作的环境，但我们修改了输出以生成每个节点的 $|\mathcal{A}|$ 个动作的可能性。他们的架构与我们的相似，但他们的方法使用DQN而不是PPO。它被包括在内，以比较这两种RL策略在此问题上的效用。表2报告了所有500个回合的平均奖励，已归一化，使得1.0是一个理论上完美的分数，如果没有任何主机被攻破。

表2：随机环境下的平均得分（越高越好）

	$ \mathcal{V} = 10$	$ \mathcal{V} = 20$	$ \mathcal{V} = 40$
标准观测	0.2511 ± 0.017	0.2310 ± 0.013	0.1683 ± 0.010
图观测	0.2090 ± 0.005	0.3037 ± 0.013	0.2047 ± 0.013
Doorman 等人 (2022)	0.7379 ± 0.011	-0.0385 ± 0.011	-0.0300 ± 0.007
归纳学习	0.4246 ± 0.011	-0.1327 ± 0.000	-0.0966 ± 0.000
朴素归纳学习	0.2681 ± 0.036	0.5439 ± 0.012	0.9308 ± 0.004
注意力归纳	0.8167 ± 0.009	0.5634 ± 0.012	0.9955 ± 0.002

毫不奇怪，归纳模型无法泛化到新环境。它很可能在训练过程中过拟合了其观察到的节点顺序。朴素模型和基于注意力的归纳模型的成绩都超过了表格方法的二倍以上。与其他基于GNN的技术相比，当环境变得更加复杂时，该模型泛化到新图的能力减弱。尽管Doorman等人（2022）的模型在训练期间取得了与我们的模型相当的成绩，但在新图上评估时，它们却无法泛化。这可能是由于PPO具有更强的探索能力，使其能够在比DQN（De La Fuente & Guerra, 2024）更复杂的环境中找到更优的策略。也可能是由于先前工作的模型在读取函数中选择的求和方式未能泛化到密度不同的图，导致过度平滑。我们还观察到，当 $|\mathcal{V}| = 40$ 时，基于注意力的模型似乎找到了一个近乎完美的策略。我们假设，由于它在更复杂的环境中训练，训练期间接收到的观察结果更加多样化，这迫使模型找到一个更通用的策略，而小环境中的模型则更容易过拟合。

²我们选择这项工作而不是Ridley (2018)，因为它在领域泛化方面有额外的关注。

To further demonstrate the generalization of the policies our models learn, we evaluate the high-scoring $|\mathcal{V}| = 40$ models in environments with different numbers of nodes than they were trained in. Unlike the prior work, which cannot generalize to different environment sizes, our approach is fully inductive. As before, we evaluated the model without retraining 10 times on 50 randomly sampled graphs for each environment size. Figure 3 plots the scores of the naive and attention-based models in these different environments.

Both models, even in the largest environment we tested, still more than double the scores of the Euclidean models in the simplest environments. The attention-based model achieves higher scores overall: its strategy remains near-perfect in environments where $|\mathcal{V}| \leq 50$. However, it is more sensitive to the growing complexity of larger environments than the naive model is. While the naive model has slightly lower scores than the attention-based model, it appears that its simpler design allowed it to remain generalizable for much more complex environments. Its score also decays as complexity increases but at a much lower rate than the attention-based model. These results are likely a result of the No Free Lunch theorem (Wolpert & Macready, 1997), which states that any increase in a model’s performance in one problem domain must be accounted for by a deficit elsewhere. These results seem to indicate that the cost of good model generality may be lower performance in easier regions of the environment-space, but consistent performance across more of the space.

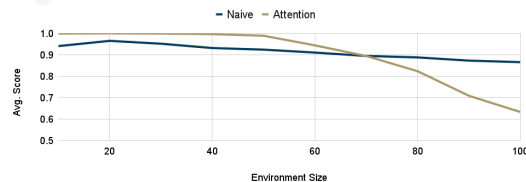


Figure 3: Scores attained by the $|\mathcal{V}| = 40$ GNN models on graphs of different sizes.

5.2 CAGE-2: A SIMULATED ENTERPRISE NETWORK

The CAGE Challenge-2 (CC2) was a reinforcement learning contest that aimed to “support the development of AI tactics, techniques, and procedures...for autonomous cyber operations” (CAGE, 2022). In this scenario, the red agent is trying to reach and compromise a specific, important host called OpServer0. CC2 provides two heuristic-based red agents: B-Line, and Meander. The B-Line agent knows the fastest route from its starting machine to its target, OpServer0, and will take the same path of lateral movements through the network with little variation. The Meander agent is slower, performing a breadth-first search through each subnet before moving to the next subnet. Specifics about the state-action space and reward function are provided in the Appendix.

Training: All GNN models are trained using the same hyperparameters and settings. For each episode, a red agent is selected randomly with even probability. We then simulate 100 timesteps. After 100 episodes, the agent’s weights are updated according to the PPO algorithm (Schulman et al., 2017). The agent only uses 4 epochs per update and otherwise uses the same hyperparameters as StableBaselines3 (Raffin et al., 2021). Both the actor and critic networks use two-layer GCNs with 256- and 64-dimensional layers.³ The attention-pooling models use a 256-dimensional global vector with mean-pooling. All models were trained for 100k, 100-step episodes.

Default Game: Each agent was evaluated on 100 episodes with lengths $\{30, 50, 100\}$ against one of the heuristic red agents. The average score of these episodes is reported in Table 3. We compare our models to the top two performers from the original competition: *Cardiff* (Hannay, 2022), an HPPO approach which trains expert agents against the B-line and Meander red agents, and heuristically decides which model to use, and *Keeping it RL* (Wolk et al., 2022) an ensemble-of-ensembles of PPO models trained against different red agents. These models outperform ours in the competition evaluation, but not by a wide margin.

As expected, because the evaluation environment is the same as the training environment, the transductive model achieves the best average score of our models. However, the attention-based model

³An ablation study on this hyperparameter is provided in the Appendix.

为了进一步展示我们模型学习的策略的泛化能力，我们在节点数量与我们训练时不同的环境中评估了高分的 $|\mathcal{V}| = 40$ 模型。与先前工作不同，后者无法泛化到不同大小的环境，我们的方法是完全归纳的。与之前一样，我们对每个环境大小在50个随机采样的图上重新训练10次模型。图3绘制了朴素模型和基于注意力的模型在这些不同环境中的成绩。

两种模型，即使在我们在测试中最大的环境中，其得分仍然比欧几里得模型在简单环境中的得分高出两倍以上。基于注意力的模型总体上实现了更高的得分：其策略在 $|\mathcal{V}| \leq 50$ 的环境中保持近乎完美。然而，它对更大环境复杂性的增长比朴素模型更敏感。虽然朴素模型的得分略低于基于注意力的模型，但其简单的的设计似乎使其能够保持对更复杂环境的泛化能力。随着复杂性的增加，它的得分也会下降，但下降速率远低于基于注意力的模型。这些结果可能是“没有免费午餐”定理 (Wolpert & Macready, 1997) 的结果，该定理指出，模型在某一问题域的性能提高必须由其他方面的缺陷来弥补。这些结果表明，良好的模型泛化能力的成本可能是环境空间中较易区域性能的降低，但在更大部分空间中保持一致的性能。

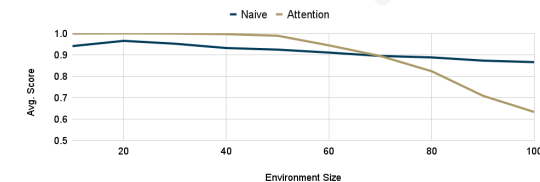


图3: 不同规模图上 $|\mathcal{V}| = 40$ GNN模型的得分。

5.2 CAGE-2: 一个模拟的企业网络

CAGE挑战赛-2 (CC2) 是一项强化学习竞赛，旨在“支持人工智能战术、技术和程序...用于自主网络作战” (CAGE, 2022年)。在这个场景中，红色代理试图到达并攻陷一个特定的、重要的主机，称为OpServer0。CC2提供了两个基于启发式的红色代理：B-Line和Meander。B-Line代理知道从其起始机器到目标OpServer0的最快路线，并且会以几乎不变的路径在网络中进行横向移动。Meander代理较慢，在移动到下一个子网之前，会对每个子网进行广度优先搜索。状态-动作空间和奖励函数的详细信息在附录中提供。

训练: 所有GNN模型都使用相同的超参数和设置进行训练。在每个回合中，以相等的概率随机选择一个红色代理。然后模拟100个时间步。在100个回合后，根据PPO算法 (Schulman等人, 2017年) 更新代理的权重。代理每次更新只使用4个epoch，否则使用与StableBaselines3 (Raffin等人, 2021年) 相同的超参数。演员网络和评价网络都使用两层GCN，维度分别为256和64。³ 注意力池化模型使用256维的全局向量，并采用均值池化。所有模型都训练了100k个、每个回合100个时间步的样本。

默认游戏: 每个代理都在 100 个长度为 $\{30, 50, 100\}$ 的回合中与其中一个启发式红色代理进行评估。这些回合的平均分数报告在表 3 中。我们将我们的模型与原始竞赛中排名前两位的表现者进行比较：Cardiff (Hannay, 2022 年)，一种 HPPO 方法，该方法通过针对 B 线和 Meander 红色代理训练专家代理，并启发式地决定使用哪个模型，以及 Keeping it RL (Wolk 等人, 2022 年)，这是一个由针对不同红色代理训练的 PPO 模型组成的集成。这些模型在竞赛评估中优于我们的模型，但优势并不明显。

正如预期，由于评估环境和训练环境相同，归纳模型在我们的模型中取得了最佳平均分。然而，基于注意力的模型

³关于此超参数的消融研究提供在附录中。

Table 3: Scores on the CC2 Environment (smaller is better)

	Total	30 Steps		50 Steps		100 Steps	
		B-Line	Meander	B-Line	Meander	B-Line	Meander
Cardiff	-54.57 $\pm$ 0.43	-3.47	-5.64	-6.41	-8.69	-13.76	-16.60
Keeping it RL	-56.90 $\pm$ 0.59	-3.48	-6.47	-5.85	-10.33	-11.39	-19.38
Transductive	-57.30 $\pm$ 0.65	-3.58	-6.25	-6.59	-9.86	-14.09	-16.94
Naive Inductive	-62.08 $\pm$ 0.63	-4.16	-7.20	-7.21	-10.90	-15.15	-17.45
Attention Inductive	-60.14 $\pm$ 0.66	-4.39	-6.99	-7.68	-9.80	-14.80	-16.48

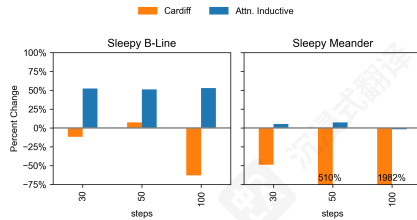


Figure 4: Evaluation against unseen red agents

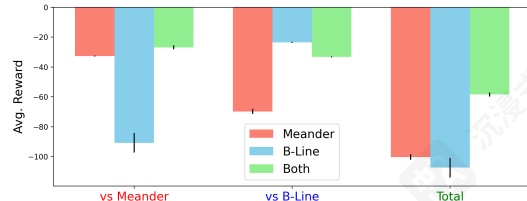


Figure 5: Evaluation of models trained against a single red agent tested against both red agents

scores almost as high in several instances, and even better in the 100-step game against Meander. Additionally, the difference in performance between the inductive and transductive models is minor.

Adversary Generalization: While the top models are very good at this specific task, they are also very brittle and overfit to the environment. In a realistic scenario, we expect the adversary to act in unpredictable ways. We create two new red agents: Sleepy-Meander, and Sleepy-B-Line. These are slower red agents; the only difference in their strategy is that at every turn they have a 50% chance of selecting no-op instead of the move the non-sleepy agent would have taken. Compare this to training against automated attacker agents, and the defense agent encountering a slower, human attacker for the first time. We evaluate our transductive model and the CardiffUni model against these new agents with no retraining. The results in Figure 4 show that this minor, realistic perturbation caused the Cardiff agent to experience a 1,982% decrease in its score in the worst case.

In comparison, utilizing relational inductive bias allows our agents to learn more generalizable policies, which is evident in how they can take advantage of the weaknesses of new attackers. For example, because the Sleepy B-Line agent is 50% slower, our model achieves a 50% better score. Against the slower meander agent, the Cardiff HPPO agent selects the expert policy for B-Line and suffers immensely. On the other hand, our agent applies its more universal policy that it learned from graph analysis and scores slightly better, or about equal to before. These results show that relational inductive bias allows for generalization to new attackers; next, we will evaluate our agents’ ability to generalize to new environments.

In another experiment, we trained our blue agents only against a single red agent before testing them against both red agents. The results of this study are shown in Figure 5. We found that against unseen agents, our models perform comparably to the HPPO agent evaluated by Wolk et al. (2022). Interestingly, the agent trained against only Meander scores slightly lower against the Meander agent, compared to the baseline agent that was trained against both red agents. These results suggest that the more general policy the default agent learned to defend against both red agents is stronger than the locally optimal policy the Meander-specialist discovered to defend against a single agent.

Domain Generalization: In addition to new adversaries, it is important to evaluate how agents behave when faced with new environments. This concern about generalizability was shared by Wolk et al. (2022), so they devised 3 new scenarios to evaluate their models in new environments. Scenarios 3 and 4 shuffle the order that hosts are indexed in two of the subnets; Scenario 5 adds paths from each machine in one subnet to two hosts in another subnet. These changes are made to the configuration file that generates the environment and are not explicitly communicated to the

表3: CC2环境得分（越低越好）

	总计	30步		50步		100步	
		B线	蜿蜒	B-Line	蜿蜒	B-Line	蜿蜒
卡迪夫	-54.57 $\pm$ 0.43	-3.47	-5.64	-6.41	-8.69	-13.76	-16.60
保持真实	-56.90 $\pm$ 0.59	-3.48	-6.47	-5.85	-10.33	-11.39	-19.38
转导式	-57.30 $\pm$ 0.65	-3.58	-6.25	-6.59	-9.86	-14.09	-16.94
朴素归纳式	-62.08 $\pm$ 0.63	-4.16	-7.20	-7.21	-10.90	-15.15	-17.45
注意力归纳	-60.14 $\pm$ 0.66	-4.39	-6.99	-7.68	-9.80	-14.80	-16.48

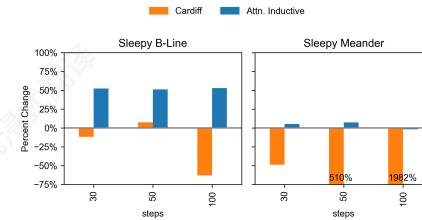


图4：对未见过的红方智能体进行评估

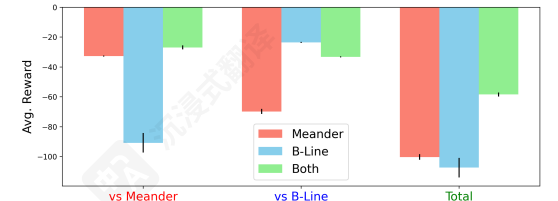


图5：针对单个红方智能体训练的模型，在同时面对两个红方智能体时进行评估

在几个实例中的得分几乎一样高，甚至在对抗Meander的100步游戏中表现更好。此外，归纳模型和归纳模型之间的性能差异很小。

对抗泛化：虽然顶尖模型在这个特定任务上表现非常出色，但它们也非常脆弱，并且过度拟合了环境。在现实场景中，我们预期对手会以不可预测的方式行动。我们创建了两个新的红方智能体：Sleepy-Meander 和 Sleepy-B-Line。这些是速度较慢的红方智能体；它们策略上的唯一区别在于，在每一步都有50%的概率选择无操作，而不是非睡眠智能体本会采取的行动。将其与针对自动化攻击智能体进行训练的情况进行比较，以及防御智能体首次遇到较慢的人类攻击者。我们使用这些新智能体对转导模型和CardiffUni模型进行评估，且不进行重新训练。图4中的结果表明，这种微小、现实的扰动导致Cardiff智能体在最坏情况下的得分下降了1,982%。

相比之下，利用关系归纳偏差使我们的智能体能够学习更具泛化能力的策略，这一点体现在它们如何利用新攻击者的弱点。例如，由于 Sleepy B-Line 智能体速度较慢 50%，我们的模型得分提高了 50%。面对较慢的 meander 智能体，Cardiff HPPO 智能体选择 B-Line 的专家策略并遭受巨大损失。另一方面，我们的智能体应用了从图分析中学习到的更通用的策略，得分略好，或与之之前相当。这些结果表明关系归纳偏差允许泛化到新攻击者；接下来，我们将评估我们的智能体泛化到新环境的能力。

在另一个实验中，我们仅用单个红色智能体训练我们的蓝色智能体，然后再用两个红色智能体测试它们。这项研究的结果如图 5 所示。我们发现，面对未见过的智能体，我们的模型表现与 Wolk 等人（2022）评估的 HPPO 智能体相当。有趣的是，仅与 Meander 训练的智能体在 Meander 智能体上的得分略低于与两个红色智能体训练的基线智能体。这些结果表明，默认智能体学习到的更通用策略（用于防御两个红色智能体）比 Meander 专家发现的局部最优策略（用于防御单个智能体）更强。

领域泛化：除了新的攻击者，还需要评估智能体在面对新环境时的行为。关于泛化性的担忧被 Wolk等人（2022年）所共享，因此他们设计了3个新场景来评估他们的模型在新环境中的表现。场景3和4打乱了两个子网中主机索引的顺序；场景5在每个子网中的每台机器添加了到另一个子网的两个主机的路径。这些更改是在生成环境的配置文件中进行的，并且没有明确地传达给智能体。

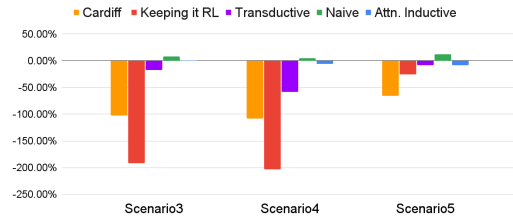


Figure 6: Change in average score under new scenarios proposed by “Keeping it RL” Wolk et al. (2022)

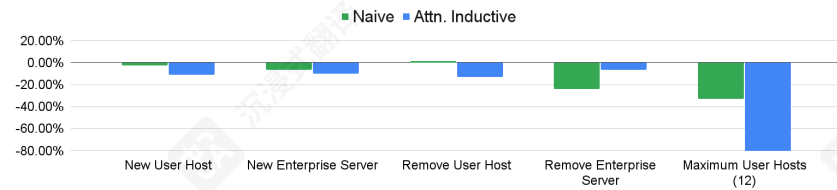


Figure 7: Change in inductive agents’ scores in new environments with different numbers of hosts.

agents. For all generalization experiments, we evaluate agents with the parameters learned from the default CC2 environment without retraining.

Figure 6 shows how different agents respond in these new scenarios. We note that even our transductive model out-performs the prior works, highlighting the utility of relational inductive bias. We further observe that, as the theoretical backing predicts, the inductive models are unaffected by the scenarios that change the ordering of nodes.

The scenarios proposed by Wolk et al. (2022) are a good starting point, but we feel that their changes do not go far enough. In addition to these scenarios, we also evaluate what happens when every index in the graph is perturbed. We observed that both inductive models’ scores changed by $< 2\%$ for the index perturbation experiments, which was well within the standard error we observed in the previous experiments. However, the transductive model’s average score dropped to -1,950.72.⁴ From this, we conclude that the transductive model is also overfit.

Finally, we evaluate several new scenarios. These scenarios each involve a different number of hosts than were present in the training graph, so the tabular methods that we evaluated previously and the transductive model cannot run without retraining. Figure 7 shows how changing the environment affects the inductive agents’ abilities to defend the network. In the first four scenarios, we simply add or delete a node from a subnet. These minor perturbations in the network affect our agents very little. In both instances where a host is added, the agents’ scores decrease slightly, but this is expected, as the attack surface increases with every additional host. Removing an enterprise server negatively affects the naive model, while having very little effect on the self-attention model. The naive model has only a local view of each node, so it does not know when to prioritize other subnets. As a result, it rarely attempts to defend the user subnet, opting instead to defend the Enterprise subnetwork. This inability to shift its attention likely explains the score decrease.

In the last scenario, where the user subnetwork is filled, both agents have difficulty defending the network, as the attack surface is nearly doubled: 8 new users are added. Like the Yawning Titan experiments, the increase in the size of the environment affects the self-attention network more than the inductive network, which highlights the difficulty and importance of balancing good scores in simpler areas of the problem space with generality across regions with greater complexity.

⁴For reference, selecting actions randomly will produce a score of 1999.41.

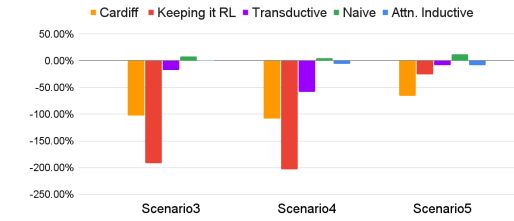


图6：“保持RL” Wolk等人（2022）提出的新场景下的平均分数变化

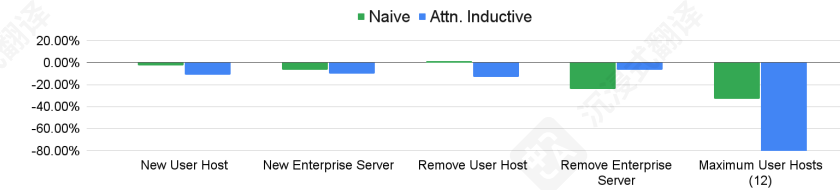


图7：不同主机数量新环境中的归纳代理分数变化

对于所有泛化实验，我们都使用从默认CC2环境中学习到的参数来评估智能体，而无需重新训练。

图6显示了不同代理在这些新场景中的响应方式。我们注意到，即使我们的传导模型也比以往的工作表现更好，突出了关系归纳偏差的效用。我们进一步观察到，正如理论支持所预测的那样，归纳模型不受节点顺序变化的场景影响。

Wolk等人（2022）提出的新场景是一个不错的起点，但我们认为他们的改变还不够彻底。除了这些场景外，我们还评估了当图中每个索引扰动时会发生什么。我们观察到，在索引扰动实验中，两个归纳模型的分数变化了 $< 2\%$ ，这在我们之前的实验中观察到的标准误差范围内。然而，传导模型的平均分数降至-1,950.72.⁴由此，我们得出结论，传导模型也存在过拟合。

最后，我们评估了几个新的场景。这些场景中的主机数量与训练图中的不同，因此我们之前评估的表格方法和传导模型都需要重新训练才能运行。图7展示了环境变化如何影响归纳代理防御网络的能力。在前四个场景中，我们只是从子网中添加或删除一个节点。这些网络中的微小扰动对代理影响很小。在添加主机的两种情况下，代理的得分略有下降，但这可以预期，因为每增加一个主机，攻击面都会增加。移除企业服务器对朴素模型产生负面影响，而对自注意力模型影响很小。朴素模型只能看到每个节点的局部视图，因此不知道何时优先考虑其他子网。结果，它很少尝试防御用户子网，而是选择防御企业子网。这种无法转移注意力的能力很可能解释了得分下降。

在最后一个场景中，当用户子网络被填满时，两个代理都难以防御网络，因为攻击面几乎翻了一番：增加了8个新用户。与Yawning Titan实验类似，环境规模的增加对自注意力网络的影响大于归纳网络，这突出了在问题空间中较简单的区域平衡良好分数与在具有更大复杂性的区域保持泛化能力的难度和重要性。

⁴参考而言，随机选择动作会产生1999.41的分数。

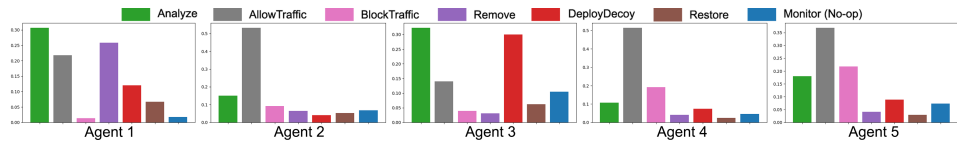


Figure 8: Action distributions of five CC4 agents. Each was trained independently, but interestingly, agents defending similar networks, e.g., Agents 2, 4, and 5 found similar strategies. Agent 1 defended the largest subnet, where it learned to analyze more frequently for better information about the large network it defended, while the latter four focused on enforcing firewall rules.

5.3 CAGE-4: MULTI-AGENT REINFORCEMENT LEARNING

The CAGE Challenge-4 (CC4) (TTCP CAGE Working Group, 2023) extends CC2 into a Multi-Agent Reinforcement Learning (MARL) task. Now, multiple agents each defend one of five segmented networks, with actions and objectives similar to CC2. One key difference for this environment is its randomly initialized topology. During each episode, subnets are generated with 1-6 servers and 3-10 user hosts. Because networks are variably sized, prior transductive and tabular-based solutions will not work.

The action space for this environment is similar to CC2. Some of the node-level actions from CC2 are merged into a single action, and there are two new edge-level actions: AllowTraffic and BlockTraffic, which create or delete edges between subnet nodes. Specifics about the state-action space and reward function are in the supplemental material.

We train five independent instances of our self-attention inductive model in this environment using the same configuration as in CC2 to measure our approach’s applicability to MARL tasks. Each agent independently learns a unique policy for the subnet it defends. We illustrate the action distribution used by each agent in Figure 8.

Our approach was the highest-performing non-heuristic agent submitted to the CC4 competition and finished in fifth place overall (Kiely et al., 2025). This result highlights the wide applicability of graph-based RL in fields beyond simple two-player games. Our agent’s ability to adapt to variable network sizes and topologies allowed us to apply it to this new challenge with only minimal changes to accommodate the new action space. In a metareview of the competition, the competition runners identified the inability to adapt to the random initialization of the environment as the main detriment for other MARL approaches that were submitted (Kiely et al., 2025). Representing actions as functions upon nodes and edges allowed our approach to naturally adapt to this challenge. These results show great promise for future research into multi-agent reinforcement learning with relational inductive bias.

6 DISCUSSION

Limitations. In this work we evaluate our approach on relatively small graphs. In all experiments, the bottleneck for throughput is the environment, rather than the model. However, with larger networks, more optimizations may be required to process observations at a reasonable speed. Another limitation is the leap from sim-to-real. Further engineering is required to address how the simplistic actions that agents take would be translated into real-world cybersecurity rules. Additionally, further analysis needs to be done to determine the utility of ACD agents, and how they would impact the humans using the networks they defend.

Future Work. We only consider actions that manifest as node-level edits, with the exception of some simple edge-level actions in CC4. Future work on efficient representation of edge- and global-level actions within a graph-based RL framework is a promising next step. Additionally, future work could include new analysis on how to better implement intra-agent communication in MARL, and studies into performance improvements.

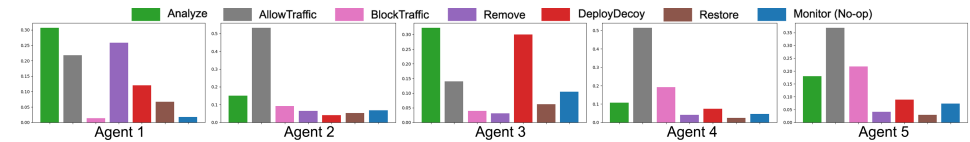


图8: 五个CC4智能体的动作分布。每个智能体都是独立训练的，但有趣的是，防御相似网络的智能体（例如智能体2、4和5）发现了相似的策略。智能体1防御了最大的子网，它学会了更频繁地分析，以获取有关其防御的大型网络更好的信息，而其他四个智能体则专注于执行防火墙规则。

5.3 CAGE-4: 多智能体强化学习

CAGE挑战赛-4 (CC4) (TTCP CAGE工作组, 2023年) 将CC2扩展为多智能体强化学习 (MARL) 任务。现在，多个代理各自防御五个分段网络中的一个，其动作和目标与CC2类似。这个环境的一个关键区别是其随机初始化的拓扑结构。在每个回合中，子网会生成1-6个服务器和3-10个用户主机。由于网络规模可变，先前的基于转导和表格的解决方案将无法工作。

此环境的动作空间与CC2类似。CC2的一些节点级动作被合并为一个动作，并且有两个新的边缘级动作：AllowTraffic和BlockTraffic，它们在子网节点之间创建或删除边。状态-动作空间和奖励函数的详细信息在补充材料中。

我们使用与CC2相同的配置，在此环境中训练了五个独立的自注意力归纳模型实例，以衡量我们的方法对MARL任务的适用性。每个智能体独立学习其防御子网的唯一策略。我们在图8中说明了每个智能体使用的动作分布。

我们的方法是在CC4竞赛中提交的非启发式智能体中表现最佳的，最终获得第五名 (Kiely等人, 2025年)。这一结果突显了基于图的强化学习在超越简单双人游戏的领域中的广泛应用。我们的智能体能够适应可变网络大小和拓扑结构，使我们能够仅通过少量修改以适应新的动作空间来应用它。在竞赛的元综述中，竞赛组织者指出，其他提交的MARL方法的主要缺陷是无法适应环境的随机初始化 (Kiely等人, 2025年)。将动作表示为作用于节点和边上的函数，使我们的方法能够自然地适应这一挑战。这些结果表明，在具有关系归纳偏差的多智能体强化学习未来研究中，前景广阔。

6 讨论

局限性。在这项工作中，我们在相对较小的图上评估了我们的方法。在所有实验中，吞吐量的瓶颈是环境，而不是模型。然而，对于更大的网络，可能需要更多的优化来以合理的速度处理观察结果。另一个局限性是从模拟到现实的跨越。需要进一步工程化来解决智能体采取的简单动作如何转化为现实世界网络安全规则的问题。此外，还需要进行进一步分析，以确定ACD智能体的效用，以及它们将如何影响使用它们防御网络的 humans。

未来工作。我们仅考虑表现为节点级编辑的操作，CC4中的一些简单边级操作除外。在基于图的强化学习框架内，对边级和全局级操作进行高效表示的未来工作是一个有前景的下一步。此外，未来工作可以包括对如何在MARL中更好地实现智能体间通信的新分析，以及性能改进的研究。

7 CONCLUSION

As automated cyber defense becomes a more plausible option for businesses and governments, the high cost of retraining agents is an important constraint to consider. We have demonstrated that our proposed agent can generalize and defend never-before-seen networks from attackers with novel behavior. Because the agent understands the network as a graph and views its actions as graph edits, its policy is less sensitive to environmental perturbation. We show that our graph-based approach outperforms top RL approaches in many environments by a wide margin in the face of slight and major environmental perturbations. Our results show that relational inductive bias is a powerful tool for improving agents' generalizability, and a step toward ACD in real-world systems.

REFERENCES

- Paul Almasan, José Suárez-Varela, Krzysztof Rusek, Pere Barlet-Ros, and Albert Cabellos-Aparicio. Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case. *Computer Communications*, 196:184–194, 2022.
- Andy Applebaum, Camron Dennler, Patrick Dwyer, Marina Moskowitz, Harold Nguyen, Nicole Nichols, Nicole Park, Paul Rachwalski, Frank Rau, Adrian Webster, et al. Bridging automated to autonomous cyber defense: Foundational analysis of tabular q-learning. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, pp. 149–159, 2022.
- Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- Lashon B Booker and Scott A Musman. A model-based, decision-theoretic perspective on automated cyber response. *arXiv preprint arXiv:2002.08957*, 2020.
- CAGE. TTCP CAGE challenge 2. In *AAAI-22 Workshop on Artificial Intelligence for Cyber Security (AICS)*, 2022.
- Hasan Cam. Cyber resilience using autonomous agents and reinforcement learning. In *Artificial intelligence and machine learning for multi-domain operations applications II*, volume 11413, pp. 219–234. SPIE, 2020.
- Xinzhong Chai, Yasen Wang, Chuanxu Yan, Yuan Zhao, Wenlong Chen, and Xiaolei Wang. Dq-motag: deep reinforcement learning-based moving target defense against ddos attacks. In *2020 IEEE Fifth International Conference on Data Science in Cyberspace (DSC)*, pp. 375–379. IEEE, 2020.
- Ankur Chowdhary, Dijiang Huang, Abdulhakim Sabur, Neha Vadnere, Myong Kang, and Bruce Montrose. Sdn-based moving target defense using multi-agent reinforcement learning. In *Proceedings of the first International Conference on Autonomous Intelligent Cyber defense Agents (AICA 2021), Paris, France*, pp. 15–16, 2021.
- Josh Collyer, Alex Andrew, and Duncan Hodges. Acd-g: Enhancing autonomous cyber defense agent generalization through graph embedded network representation. In *International Conference on Machine Learning*, 2022.
- Neil De La Fuente and Daniel A Vidal Guerra. A comparative study of deep reinforcement learning models: Dqn vs ppo vs a2c. *arXiv preprint arXiv:2407.14151*, 2024.
- Christoffel Doorman, Victor-Alexandru Darvariu, Stephen Hailes, and Mirco Musolesi. Dynamic network reconfiguration for entropy maximization using deep reinforcement learning. In *Learning on Graphs Conference*, pp. 49–1. PMLR, 2022.
- Taha Egthesad, Yevgeniy Vorobeychik, and Aron Laszka. Adversarial deep reinforcement learning based adaptive moving target defense. In *Decision and Game Theory for Security: 11th International Conference, GameSec 2020, College Park, MD, USA, October 28–30, 2020, Proceedings 11*, pp. 58–79. Springer, 2020.

7 结论

随着自动化网络防御成为企业和政府更可行的选择，重新训练智能体的高成本是一个重要的约束因素。我们已经证明，我们提出的智能体可以泛化并防御从未见过的新网络攻击者。由于智能体将网络视为图，并将其操作视为图编辑，因此其策略对环境扰动不太敏感。我们表明，在轻微和重大环境扰动面前，我们的基于图的方法在许多环境中以很大优势优于顶级强化学习方法。我们的结果表明，关系归纳偏差是提高智能体泛化能力的有力工具，也是迈向现实系统中自动化网络防御的一步。

参考文献

- Paul Almasan, José Suárez-Varela, Krzysztof Rusek, Pere Barlet-Ros, and Albert Cabellos-Aparicio. Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case. *Computer Communications*, 196:184–194, 2022.
- Andy Applebaum, Camron Dennler, Patrick Dwyer, Marina Moskowitz, Harold Nguyen, Nicole Nichols, Nicole Park, Paul Rachwalski, Frank Rau, Adrian Webster, et al. Bridging automated to autonomous cyber defense: Foundational analysis of tabular q-learning. In *Proceedings of the 15th ACM Workshop on Artificial Intelligence and Security*, pp. 149–159, 2022.
- Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261*, 2018.
- Lashon B Booker and Scott A Musman. A model-based, decision-theoretic perspective on automated cyber response. *arXiv preprint arXiv:2002.08957*, 2020.
- CAGE. TTCP CAGE challenge 2. In *AAAI-22 Workshop on Artificial Intelligence for Cyber Security (AICS)*, 2022.
- Hasan Cam. Cyber resilience using autonomous agents and reinforcement learning. In *Artificial intelligence and machine learning for multi-domain operations applications II*, volume 11413, pp. 219–234. SPIE, 2020.
- Xinzhong Chai, Yasen Wang, Chuanxu Yan, Yuan Zhao, Wenlong Chen, and Xiaolei Wang. Dq-motag: deep reinforcement learning-based moving target defense against ddos attacks. In *2020 IEEE Fifth International Conference on Data Science in Cyberspace (DSC)*, pp. 375–379. IEEE, 2020.
- Ankur Chowdhary, Dijiang Huang, Abdulhakim Sabur, Neha Vadnere, Myong Kang, and Bruce Montrose. Sdn-based moving target defense using multi-agent reinforcement learning. In *Proceedings of the first International Conference on Autonomous Intelligent Cyber defense Agents (AICA 2021), Paris, France*, pp. 15–16, 2021.
- Josh Collyer, Alex Andrew, and Duncan Hodges. Acd-g: Enhancing autonomous cyber defense agent generalization through graph embedded network representation. In *International Conference on Machine Learning*, 2022.
- Neil De La Fuente and Daniel A Vidal Guerra. A comparative study of deep reinforcement learning models: Dqn vs ppo vs a2c. *arXiv preprint arXiv:2407.14151*, 2024.
- Christoffel Doorman, Victor-Alexandru Darvariu, Stephen Hailes, and Mirco Musolesi. Dynamic network reconfiguration for entropy maximization using deep reinforcement learning. In *Learning on Graphs Conference*, pp. 49–1. PMLR, 2022.
- Taha Egthesad, Yevgeniy Vorobeychik, and Aron Laszka. Adversarial deep reinforcement learning based adaptive moving target defense. In *Decision and Game Theory for Security: 11th International Conference, GameSec 2020, College Park, MD, USA, October 28–30, 2020, Proceedings 11*, pp. 58–79. Springer, 2020.

Paul Erdős, Alfréd Rényi, et al. On the evolution of random graphs. *Publ. math. inst. hung. acad. sci*, 5(1):17–60, 1960.

Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

Myles Foley, Chris Hicks, Kate Highnam, and Vasilios Mavroudis. Autonomous network defence using reinforcement learning. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pp. 1252–1254, 2022.

Rohit Gangupantulu, Tyler Cody, Abdul Rahma, Christopher Redino, Ryan Clark, and Paul Park. Crown jewels analysis using reinforcement learning with attack graphs. In *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 1–6. IEEE, 2021.

Chungang Gao and Yongjie Wang. Reinforcement learning based self-adaptive moving target defense against ddos attacks. In *Journal of Physics: Conference Series*, volume 1812, pp. 012039. IOP Publishing, 2021.

Yazhuo Gao, Guomin Zhang, and Changyou Xing. A multiphase dynamic deployment mechanism of virtualized honeypots based on intelligent attack path prediction. *Security and Communication Networks*, 2021:1–15, 2021.

Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural computation*, 4(1):1–58, 1992.

Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

Jessica B Hamrick, Kelsey R Allen, Victor Bapst, Tina Zhu, Kevin R McKee, Joshua B Tenenbaum, and Peter W Battaglia. Relational inductive bias for physical construction in humans and machines. *arXiv preprint arXiv:1806.01203*, 2018.

John Hannay. CardiffUni CAGE2 submission. <https://github.com/john-cardiff/-cyborg-cage-2>, 2022.

Jaromír Janisch, Tomáš Pevný, and Viliam Lisý. Symbolic relational deep reinforcement learning based on graph neural networks. *arXiv preprint arXiv:2009.12462*, 2020.

Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. *Advances in neural information processing systems*, 30, 2017.

Mitchell Kiely, Metin Ahiskali, Etienne Borde, Benjamin Bowman, David Bowman, Dirk van Bruggen, KC Cowan, Prithviraj Dasgupta, Erich Devendorf, Ben Edwards, et al. Exploring the efficacy of multi-agent reinforcement learning for autonomous cyber defence: A cage challenge 4 perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 28907–28913, 2025.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.

Guillaume Lachaud, Patricia Conde-Cespedes, and Maria Trocan. Comparison between inductive and transductive learning in a real citation network using graph neural networks. In *2022 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, pp. 534–540. IEEE, 2022.

Tom M Mitchell. The need for biases in learning generalizations. *Rutgers Technical Report*, CBM-TR 5-110, 1980.

Paul Erdős, Alfréd Rényi, et al. On the evolution of random graphs. *Publ. math. inst. hung. acad. sci*, 5(1):17–60, 1960.

Matthias Fey and Jan E. Lenssen. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

Myles Foley, Chris Hicks, Kate Highnam, and Vasilios Mavroudis. Autonomous network defence using reinforcement learning. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, pp. 1252–1254, 2022.

Rohit Gangupantulu, Tyler Cody, Abdul Rahma, Christopher Redino, Ryan Clark, and Paul Park. Crown jewels analysis using reinforcement learning with attack graphs. In *2021 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 1–6. IEEE, 2021.

Chungang Gao and Yongjie Wang. Reinforcement learning based self-adaptive moving target defense against ddos attacks. In *Journal of Physics: Conference Series*, volume 1812, pp. 012039. IOP Publishing, 2021.

Yazhuo Gao, Guomin Zhang, and Changyou Xing. A multiphase dynamic deployment mechanism of virtualized honeypots based on intelligent attack path prediction. *Security and Communication Networks*, 2021:1–15, 2021.

Stuart Geman, Elie Bienenstock, and René Doursat. Neural networks and the bias/variance dilemma. *Neural computation*, 4(1):1–58, 1992.

Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

Jessica B Hamrick, Kelsey R Allen, Victor Bapst, Tina Zhu, Kevin R McKee, Joshua B Tenenbaum, and Peter W Battaglia. Relational inductive bias for physical construction in humans and machines. *arXiv preprint arXiv:1806.01203*, 2018.

John Hannay. CardiffUni CAGE2 submission. <https://github.com/john-cardiff/-cyborg-cage-2>, 2022.

Jaromír Janisch, Tomáš Pevný, and Viliam Lisý. Symbolic relational deep reinforcement learning based on graph neural networks. *arXiv preprint arXiv:2009.12462*, 2020.

Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilkina, and Le Song. Learning combinatorial optimization algorithms over graphs. *Advances in neural information processing systems*, 30, 2017.

Mitchell Kiely, Metin Ahiskali, Etienne Borde, Benjamin Bowman, David Bowman, Dirk van Bruggen, KC Cowan, Prithviraj Dasgupta, Erich Devendorf, Ben Edwards, et al. Exploring the efficacy of multi-agent reinforcement learning for autonomous cyber defence: A cage challenge 4 perspective. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pp. 28907–28913, 2025.

Thomas N Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.

Guillaume Lachaud, Patricia Conde-Cespedes, and Maria Trocan. Comparison between inductive and transductive learning in a real citation network using graph neural networks. In *2022 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*, pp. 534–540. IEEE, 2022.

Tom M Mitchell. The need for biases in learning generalizations. *Rutgers Technical Report*, CBM-TR 5-110, 1980.

Aritran Piplai, Mike Anoruo, Kayode Fasaye, Anupam Joshi, Tim Finin, and Ahmad Ridley. Knowledge guided two-player reinforcement learning for cyber attacks and defenses. In *2022 21st IEEE International Conference on Machine Learning and Applications (ICMLA)*, pp. 1342–1349. IEEE, 2022.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.

Leonardo FR Ribeiro, Pedro HP Saverese, and Daniel R Figueiredo. struc2vec: Learning node representations from structural identity. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 385–394, 2017.

Ahmad Ridley. Machine learning for autonomous cyber defense. *The Next Wave*, 22(1):7–14, 2018.

Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

TTCP CAGE Working Group. Ttcp cage challenge 4. <https://github.com/cage-challenge/cage-challenge-4>, 2023.

Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. Graph attention networks. *stat*, 1050(20):10–48550, 2017.

Sanyam Vyas, John Hannay, Andrew Bolton, and Professor Pete Burnap. Automated cyber defence: A review. *arXiv preprint arXiv:2303.04926*, 2023.

Erich Walter, Kimberly Ferguson-Walter, and Ahmad Ridley. Incorporating deception into cyber-battlesim for autonomous defense. *arXiv preprint arXiv:2108.13980*, 2021.

Tingwu Wang, Renjie Liao, Jimmy Ba, and Sanja Fidler. Nervenet: Learning structured policy with graph neural networks. In *International conference on learning representations*, 2018.

Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8:279–292, 1992.

Shimon Whiteson, Brian Tanner, Matthew E Taylor, and Peter Stone. Protecting against evaluation overfitting in empirical reinforcement learning. In *2011 IEEE symposium on adaptive dynamic programming and reinforcement learning (ADPRL)*, pp. 120–127. IEEE, 2011.

Melody Wolk, Andy Applebaum, Camron Dennler, Patrick Dwyer, Marina Moskowitz, Harold Nguyen, Nicole Nichols, Nicole Park, Paul Rachwalski, Frank Rau, et al. Beyond cage: Investigating generalization of learned autonomous network defense policies. *arXiv preprint arXiv:2211.15557*, 2022.

David H Wolpert and William G Macready. No free lunch theorems for optimization. *IEEE transactions on evolutionary computation*, 1(1):67–82, 1997.

Cangshuai Wu, Jiangyong Shi, Yuexiang Yang, and Wenhua Li. Enhancing machine learning based malware detection model by reinforcement learning. In *Proceedings of the 8th International Conference on Communication and Network Security*, pp. 74–78, 2018.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=ryGs6iA5Km>.

Mehdi Yousefi, Nhamo Mtetwa, Yan Zhang, and Huaglory Tianfield. A reinforcement learning approach for attack graph analysis. In *2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*, pp. 212–217. IEEE, 2018.

Aritran Piplai, Mike Anoruo, Kayode Fasaye, Anupam Joshi, Tim Finin, and Ahmad Ridley. Knowledge guided two-player reinforcement learning for cyber attacks and defenses. In *2022 21st IEEE International Conference on Machine Learning and Applications (ICMLA)*, pp. 1342–1349. IEEE, 2022.

Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021. URL <http://jmlr.org/papers/v22/20-1364.html>.

Leonardo FR Ribeiro, Pedro HP Saverese, and Daniel R Figueiredo. struc2vec: Learning node representations from structural identity. In *Proceedings of the 23rd ACM SIGKDD international conference on knowledge discovery and data mining*, pp. 385–394, 2017.

Ahmad Ridley. Machine learning for autonomous cyber defense. *The Next Wave*, 22(1):7–14, 2018.

Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

TTCP CAGE Working Group. Ttcp cage challenge 4. <https://github.com/cage-challenge/cage-challenge-4>, 2023.

Petar Velickovic, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, Yoshua Bengio, et al. Graph attention networks. *stat*, 1050(20):10–48550, 2017.

Sanyam Vyas, John Hannay, Andrew Bolton, and Professor Pete Burnap. Automated cyber defence: A review. *arXiv preprint arXiv:2303.04926*, 2023.

Erich Walter, Kimberly Ferguson-Walter, and Ahmad Ridley. Incorporating deception into cyber-battlesim for autonomous defense. *arXiv preprint arXiv:2108.13980*, 2021.

Tingwu Wang, Renjie Liao, Jimmy Ba, and Sanja Fidler. Nervenet: Learning structured policy with graph neural networks. In *International conference on learning representations*, 2018.

Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8:279–292, 1992.

Shimon Whiteson, Brian Tanner, Matthew E Taylor, and Peter Stone. Protecting against evaluation overfitting in empirical reinforcement learning. In *2011 IEEE symposium on adaptive dynamic programming and reinforcement learning (ADPRL)*, pp. 120–127. IEEE, 2011.

Melody Wolk, Andy Applebaum, Camron Dennler, Patrick Dwyer, Marina Moskowitz, Harold Nguyen, Nicole Nichols, Nicole Park, Paul Rachwalski, Frank Rau, et al. Beyond cage: Investigating generalization of learned autonomous network defense policies. *arXiv preprint arXiv:2211.15557*, 2022.

David H Wolpert and William G Macready. No free lunch theorems for optimization. *IEEE transactions on evolutionary computation*, 1(1):67–82, 1997.

Cangshuai Wu, Jiangyong Shi, Yuexiang Yang, and Wenhua Li. Enhancing machine learning based malware detection model by reinforcement learning. In *Proceedings of the 8th International Conference on Communication and Network Security*, pp. 74–78, 2018.

Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=ryGs6iA5Km>.

Mehdi Yousefi, Nhamo Mtetwa, Yan Zhang, and Huaglory Tianfield. A reinforcement learning approach for attack graph analysis. In *2018 17th IEEE International Conference On Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*, pp. 212–217. IEEE, 2018.

A TRAINING CONFIGURATION

To better facilitate replication of our work, in Table 4, we provide the hyperparameters used for each model. Due to the length of time required to train each model, we did not run ablation every hyperparameter, and instead opted for default settings in most cases. In the Yawning Titan environment, we selected 5M steps as the length of training time for fair comparison with the original paper. We selected the lower hidden and embedding dimension in Yawning Titan after initial tests with parameters identical to the CC2 environment performed poorly. In both environments, we tested both max and mean pooling and selected the model that performed best.

Table 4: Model Hyperparameters

	YT	CC2/4
Hidden Dimension	64	256
Embedding Dimension	32	64
Pooling Function	Max	Mean
Actor LR	3e-4	3e-4
Critic LR	1e-3	1e-3
Episode Length	600 Steps	100 Steps
Episodes per Update (N)	10	100
Training Time	5M Steps	1M Episodes

All agents were trained on a server with an Intel Xeon Gold 6338 CPU using a maximum of 100 cores to simulate N episodes in parallel. Using a GPU would accelerate training slightly, but the main performance bottleneck was the environment simulators, which were CPU-only.

B ABLATION STUDIES

In this section, we present ablation studies on the number of parameters used by the models, and the choice of GNN. For all ablation experiments, due to the length of time required to train models for 1M episodes, we constrain training time to 100k episodes. Otherwise, all parameters are fixed, and the same as they were for the original CC2 experiments other than the hyperparameter being ablated.

Hidden Dimensions. In these experiments, in addition to changing the hidden dimension, we also kept the ratio of hidden to embedding dimensions the same (4 to 1), such that the agent with a 32-dimensional hidden dimension has an 8-dimensional embedding, and the agent with 1028 hidden dimensions has a 256-dimensional embedding. Otherwise, all other hyperparameters are the same as before. We observe that as more parameters are added to the models, their average score does increase somewhat. However, adding additional dimensions comes at a cost: as models grow larger, the time it takes to perform forward and backward passes has a steep increase. Our choice in selecting 256 as the hidden dimension was to strike a good balance between score and efficiency.

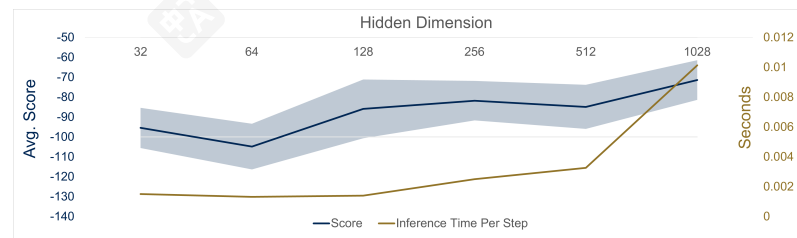


Figure 9: Average score (smaller is better) as more parameters are added to the model (standard deviation plotted as the shaded region).

Choice of GNN. To evaluate the model’s sensitivity to the choice of GNN, we evaluate models trained in the CC2 environment with several different base GNNs, namely SAGE Hamilton et al. (2017), GAT Velickovic et al. (2017), GIN Xu et al. (2019). All training parameters are the same, and models are each trained for 100k episodes. We observe that each model performs roughly as well as the others. GAT has the best average performance overall, but when the standard error is

训练配置

为更好地促进我们工作的复现，在表4中，我们提供了每个模型使用的超参数。由于训练每个模型所需的时间较长，我们没有运行每个超参数的消融实验，而是在大多数情况下选择了默认设置。在 Yawning Titan环境中，我们选择了5M步作为训练时间长度，以便与原始论文进行公平比较。在 Yawning Titan中，经过初步测试后，我们发现与CC2环境参数相同的模型表现不佳，因此选择了较低的隐藏层和嵌入维度。在两个环境中，我们都测试了最大池化和平均池化，并选择了表现最好的模型。

表4：模型超参数

	YT	CC2/4
隐藏维度	64	256
嵌入维度	32	64
池化函数	Max	Mean
Actor 学习率	3e-4	3e-4
Critic 学习率	1e-3	1e-3
回合长度	600 步	100 步
每次更新中的剧集数 (N)	10	100
训练时间	5M 步骤	1M 剧集

所有智能体均在配备 Intel Xeon Gold 6338 CPU 的服务器上训练，最多使用 100 个核心以并行模拟 N 个剧集。使用 GPU 可略微加速训练，但主要性能瓶颈是环境模拟器，这些模拟器仅支持 CPU。

B ABLATION 研究

在本节中，我们展示了模型参数数量和 GNN 选择上的消融研究。对于所有消融实验，由于训练 1M 个回合的模型需要较长时间，我们将训练时间限制为 100k 个回合。否则，所有参数均保持固定，与原始 CC2 实验相同，仅消融的超参数除外。

隐藏维度。在这些实验中，除了改变隐藏维度外，我们还保持隐藏维度与嵌入维度的比例相同（4:1），即 32 维隐藏维度的智能体对应 8 维嵌入维度，1028 维隐藏维度的智能体对应 256 维嵌入维度。其余所有超参数与之前相同。我们观察到随着模型参数的增加，其平均分数确实有所提升。然而，增加额外维度是有代价的：随着模型规模增大，前向和后向传播所需的时间会急剧增加。我们选择 256 作为隐藏维度的原因是在分数和效率之间取得了良好的平衡。

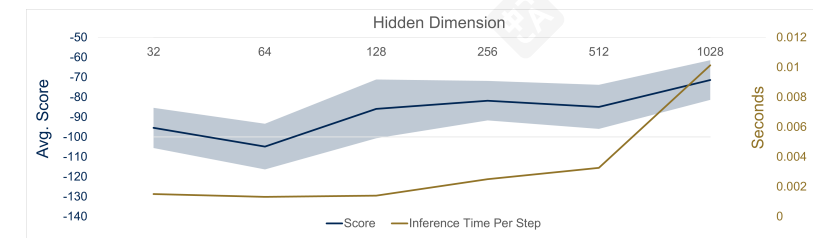


图9：随着模型参数增加，平均得分（越低越好）（标准差以阴影区域表示）。

GNN 的选择。为了评估模型对 GNN 选择敏感性的影响，我们在 CC2 环境中用几种不同的基础 GNN 训练模型，具体为 SAGE Hamilton 等人 (2017)，GAT Velickovic 等人 (2017)，GIN Xu 等人 (2019)。所有训练参数相同，每个模型均训练 100k 个回合。我们观察到每个模型的性能大致相当。GAT 的平均性能最好，但当考虑标准误差时，它仅比 SAGE 有显著性优势 ($\{v1\}$)，而其他两个模型则没有 ($\{v2\}$)。这些结果支持我们的观点，即该方法与模型无关；我们的方法对 GNN 的选择在统计上没有显著敏感性。

taken into account, it is only better than SAGE with significance ($p = 0.04$, while $p > 0.1$ for the other two models). These results support our claim that the approach is model-agnostic; our approach is not sensitive to the choice of GNN with statistical significance.

Table 5: Scores of models with different base GNNs on CC2

	Total	30 Steps		50 Steps		100 Steps	
		B-Line	Meander	B-Line	Meander	B-Line	Meander
GCN	-80.32 $\pm$ 0.99	-5.521	-8.136	-10.718	-12.541	-21.709	-21.698
SAGE	-84.23 $\pm$ 1.05	-5.458	-9.563	-8.727	-15.069	-19.232	-26.18
GAT	-77.59 $\pm$ 1.39	-5.164	-9.404	-9.373	-14.282	-16.937	-22.434
GIN	-81.44 $\pm$ 1.05	-5.809	-8.003	-9.954	-12.802	-22.692	-22.179

C ENVIRONMENT DESCRIPTIONS

The Yawning Titan environment represents nodes with two features each: one representing how likely an attacker is to be successful at compromising that node, and one representing whether they have been compromised. Additionally, the full list of edges in the graph representing the network is also available. We use the combination of node features and edge information as inputs to the GNNs that power our models. There are also three actions in this environment These actions are all listed and described in Table 6. In practice, however, we do not implement the Sleep action, as it is always better to Upgrade a node rather than do nothing.

Table 6: Yawning Titan Actions

Action	Description	Effect
Restore	Removes the attacker from the selected host. Simulates restoring from a previously saved image.	Update node features to “non-compromised” and reset the “vulnerability” feature to its initial value
Upgrade	Makes the selected host more difficult to compromise in the future. Simulating a software upgrade or patch.	Decrease the node’s “vulnerability” feature by 0.2.
Sleep	Do nothing	

The reward function is the ratio of compromised to non-compromised hosts at each time step, with a 100-point bonus for reaching 500 timesteps without allowing every host to be compromised. If every host is compromised before step 500, the agent receives a penalty of -100 points.

The CC2 environment provides highly detailed observations with a great deal of information about each host in the network. We only considered the Host, Connection (meaning open ports), File, and Subnet entities when constructing our graphs from the available data. Table 7 contains a full description of the features considered for each node type. Entries with an asterisk denote engineered features. Entries without an asterisk are directly provided in the observations emitted by the environment.

The environment allows for ten total action types, each described in greater detail in Table 8. Unlike the Yawning Titan environment, in CC2, actions may have a cost associated with them. In this environment, only the Restore action, which fully wipes a host and restores it from an image, has a cost of -1. The Analyze action provides more information about the files on a given host. The Remove action kills any user-level shells on the host. Finally there are seven different Decoy actions, each starting a honeypot process on a different port. Certain decoys are only allowed on specific operating systems, and decoys can only be created if the host is not running any other processes on the port they would occupy.

The reward function for this environment is the number of compromised hosts in the network. For each user-type host compromised, a penalty of 0.1 points is applied; compromised server-type hosts are penalized 1.0 points; if the critical asset OpServer0 is compromised, there is a penalty of 10 points. The full reward function is the sum of penalties from host compromises and the cost of the action selected by the agent.

The CC4 Environment is similar to the CC2 environment. Observations are identical between the environments with two differences. In CC4, as timesteps progress, the environment goes through

这些结果支持我们的主张，即该方法与模型无关；我们的方法对 GNN 的选择在统计上没有显著敏感性。

表5：不同基础GNN模型在CC2上的得分

	总计	30步		50步		100步	
		B-Line	蜿蜒	B线	蜿蜒	B线	蜿蜒
GCN	-80.32 $\pm$ 0.99	-5.521	-8.136	-10.718	-12.541	-21.709	-21.698
SAGE	-84.23 $\pm$ 1.05	-5.458	-9.563	-8.727	-15.069	-19.232	-26.18
GAT	-77.59 $\pm$ 1.39	-5.164	-9.404	-9.373	-14.282	-16.937	-22.434
GIN	-81.44 $\pm$ 1.05	-5.809	-8.003	-9.954	-12.802	-22.692	-22.179

C 环境描述

Yawning Titan 环境表示每个节点具有两个特征：一个表示攻击者成功入侵该节点的可能性，另一个表示该节点是否已被入侵。此外，还提供了表示网络的图中所有边的完整列表。我们使用节点特征和边信息的组合作为驱动我们模型的 GNN 的输入。在这个环境中还有三个动作。这些动作全部列于表 6 中并进行了描述。然而，在实践中，我们不实现 Sleep 动作，因为升级节点总比什么都不做强。

表 6: Yawning Titan 动作

动作	描述	效果
恢复	将攻击者从选定的主机中移除。模拟从先前保存的镜像中恢复。	将节点特征更新为“未受损”，并将“漏洞”特征重置为其初始值
升级	使选定的主机在未来更难被攻破模拟软件升级或补丁	将节点的“漏洞”特征减少0.2
休眠	什么也不做	

奖励函数是每个时间步长下受损主机与非受损主机的比率，若在500个时间步长内未让所有主机都受损，则获得100分奖励。如果所有主机在500步之前都受损，代理将受到-100分的惩罚。

CC2环境提供了高度详细的观察数据，包含网络中每个主机的丰富信息。在从可用数据构建图时，我们仅考虑了主机、连接（指开放端口）、文件和子网实体。表7包含了对每种节点类型所考虑特征的完整描述。带星号的项目表示人工设计的特征。不带星号的项目是环境发出的观察数据中直接提供的。

该环境允许十种总动作类型，每种动作的详细描述见表8。与Yawning Titan环境不同，在CC2中，动作可能具有成本。在此环境中，只有恢复动作（将主机完全擦除并从镜像中恢复）的成本为-1。分析动作提供更多关于给定主机上文件的信息。移除动作会杀死主机上的任何用户级shell。最后，有七种不同的诱饵动作，每种动作在不同的端口上启动蜜罐进程。某些诱饵仅允许在特定的操作系统上使用，且诱饵只能在主机不运行任何其他进程的端口上创建时才能创建。

该环境的奖励函数是网络中受感染的主机数量。对于每个受感染的用户类型主机，将扣除0.1分；受感染的服务器类型主机将被扣除1.0分；如果关键资产OpServer0受感染，将扣除10分。完整的奖励函数是主机受感染扣除的分数以及代理选择的行动成本的总和。

CC4 环境与 CC2 环境相似。这两个环境中的观察结果完全相同，但有两点区别。在 CC4 中，随着时间步的推进，环境会经历

Table 7: Node features in the CAGE Environments			
Node Type	Feature	Feature type	Additional information
Host	Architecture	One-hot	Applied to Op_Server0 node in CAGE-2. Means that this node has extra penalties for being compromised
	OS Distribution	One-hot	
	OS Type	One-hot	
	OS Version	One-hot	
	OS Kernel Version	One-hot	
	OS Patches	One-hot	
	Is Critical*	Boolean	
	Is User*	Boolean	
	Is Server*	Boolean	
Files	File Type	One-hot	
	File Path	One-hot	
	Version	One-hot	
	Type	One-hot	
	Vendor	One-hot	
	Density	Float	
	Signed	Boolean	
	User Permissions	int[0-7]	
	Group Permissions	int[0-7]	
	Default Permissions	int[0-7]	
Connection	Is Decoy*	Boolean	If the connection is to a decoy process running on the host
Subnet	None		Structural node to connect hosts residing in the same subnet

* Engineered features that are not provided by default

Table 8: Blue agent action space in the CAGE environments

Action	Description	Effect	Cost
Monitor	Review network traffic logs for suspicious activity. This action is taken implicitly every turn, but if selected explicitly, it functions as a no-op action.	Create edges from host nodes to connection nodes if network activity is observed.	0
Analyze	Attempt to learn it if this host has been compromised by the red agent.	Update node features if it is compromised. Add file nodes and edges connecting them to host if found during scan.	0
Remove	Attempt to remove the red agent from this machine. However, if the red agent has privilege escalated to root, this action will fail.	Update node features if successful.	0
Restore	Guarantees the red agent will be removed from this host, but causes significant disruption to the network	Remove all edges to open port nodes adjacent to the host. Reset the host node’s features to show it as not compromised.	1
Decoy	Open a port on this host to act as a honeypot for the red agent. If the red agent attempts to compromise that port, it will fail. There are 7 types of decoy actions. Each may only be used if the port it would open is not in use, and sometimes only if the host uses a specific OS.	Create a new open port node with an edge to connecting it to the host.	0
AllowTraffic**	Modifies a firewall rule to allow communication between two subnets	Creates an edge between the two subnet nodes	0
BlockTraffic**	Modifies a firewall rule to disallow communication between two subnets	Deletes an edge between the two subnet nodes	0

** Actions only available in the CC4 environment.

three distinct phases. During each phase, there are different firewall rules and communication policies that are allowed. We communicate the current phase to the agent as input to the global vector network. The second difference is that agents are each allowed to communicate 8 bits to one another. We used this to indicate the state of each subnet the agent was defending. For each subnet, the first bit represented if any host in the subnet had been compromised. We used the second bit as a check bit to determine the difference between no message (0,0) and a message of no compromise (0,1). These messages were used as features for the subnet nodes they corresponded to. Another important distinction is that agents do not have full knowledge of the network. Each agent only receives observations about the subnets it defends; their only knowledge of subnets outside of their influence is through the messages sent by other agents.

Last, the reward structure in CC4 is different from CC2. Rewards are based on the ability of green agents (representing employees) to do either local work, or access services throughout the network.

表7：CAGE环境中的节点特征			
节点类型	特征	特征类型	附加信息
Host	架构	One-hot	应用于 CAGE-2 中的 Op Server0 节点。表示该节点在遭受攻击时会有额外的惩罚
	操作系统发行版	One-hot	
	操作系统类型	One-hot	
	操作系统版本	One-hot	
	操作系统内核版本	One-hot	
	操作系统补丁	One-hot	
	是否关键*	布尔值	
	是否为用户*	布尔值	
	是否为服务器*	布尔值	
文件	文件类型	独热编码	
	文件路径	独热编码	
	版本	One-hot	
	Type	One-hot	
	供应商	One-hot	
	密度	浮点数	
	已签署	布尔值	
	用户权限	int[0-7]	
	组权限	int[0-7]	
	默认权限	int[0-7]	
连接	是否诱饵*	布尔值	如果连接到主机上运行的诱饵进程
子网	None		用于连接同一子网中的主机的结构节点

* 默认情况下未提供的工程化特征

表 8：CAGE 环境中蓝代理的动作空间

动作	描述	效果	Cost
显示器	检查网络流量日志中的可疑活动。此操作每回合都会隐式执行，但如果明确选择，则作为无操作（no-op）执行。	如果观察到网络活动，则从主机节点到连接节点创建边。如果观察到网络活动。	0
分析	如果该主机已被红方代理入侵，尝试学习它。	如果节点已受损，请更新节点功能。添加文件如果在过程中发现节点和连接它们的边，请将它们连接到主机。	0
移除	尝试从这台机器中移除红色代理。但是，如果红色代理的权限已提升至root，此操作将失败。	scan. 如果成功，请更新节点功能。	0
恢复	保证红色代理将从该主机中移除， 但会对网络造成重大干扰	移除所有与主机相邻的端口节点。重置主机节点的特征， 以显示其未受感染。	1
诱饵	在该主机上打开一个端口，作为红色代理的蜜罐。如果红色代理尝试感染该端口，它将失败。有7种诱饵操作类型。每种操作只有在它将要打开的端口未被使用时才能使用，有时还只有在主机使用特定操作系统时才能使用。	创建一个新的端口节点，并创建一条边将其连接到主机。	0
允许流量**	修改防火墙规则以允许两个子网之间的通信	在两个子网节点之间创建一条边	0
阻止流量**	修改防火墙规则以禁止两个子网之间的通信	删除两个子网节点之间的边	0

** 仅在 CC4 环境中可用。

三个不同的阶段。在每个阶段中，存在不同的防火墙规则和允许的通信策略。我们将当前阶段作为输入传递给代理，作为全局向量网络的输入。第二个区别是，每个代理都允许向彼此发送 8 位信息。我们使用它来表示代理所防御的每个子网的状态。对于每个子网，第一位表示子网中的任何主机是否已被攻破。我们使用第二位作为校验位，以区分无消息（0,0）和表示无攻破消息（0,1）。这些消息被用作它们对应的子网节点的特征。另一个重要区别是，代理对网络没有完全的了解。每个代理仅接收关于其防御的子网的观察结果；它们对自身影响范围之外的子网的了解仅通过其他代理发送的消息。

最后，CC4中的奖励机制与CC2不同。奖励基于绿色代理（代表员工）执行本地工作或访问整个网络服务的能力。

The red agent attempts to disrupt hosts, and if it is successful, when a green action tries to use it, it will fail, and the blue agent will be penalized. The exact penalty amount changes based on the current phase and the subnet of the host the green agent couldn't access.

红色代理试图破坏主机，如果成功，当绿色动作尝试使用它时，将失败，蓝色代理将被处罚。具体的处罚金额会根据当前阶段以及绿色代理无法访问的主机子网而变化。